Introduction to Quant Finance

Advanced Module

Quant Club IIT KGP

Summer of Quant

Advanced Module Study Notes

July 26, 2026

Contents

1 Algorithmic Trading **5**
- 1.1 What is Backtesting? . . . 5
  - 1.1.1 Definition . . . 5
  - 1.1.2 Why Backtest? . . . 5
  - 1.1.3 The Backtesting Principle . . . 6
  - 1.1.4 Requirements of a Good Backtest . . . 6
- 1.2 Performance Metrics for Backtesting a Trading Strategy . . . 7
  - 1.2.1 Overview . . . 7
  - 1.2.2 Cumulative Return (Absolute Return) . . . 7
  - 1.2.3 Annualised Return . . . 7
  - 1.2.4 Annualised Volatility . . . 8
  - 1.2.5 Sharpe Ratio . . . 8
  - 1.2.6 Sortino Ratio . . . 9
  - 1.2.7 Beta . . . 9
  - 1.2.8 Maximum Drawdown . . . 10
  - 1.2.9 Summary of Performance Metrics . . . 11
- 1.3 Slippage and Latency . . . 11
  - 1.3.1 What is Slippage? . . . 11
  - 1.3.2 Why Slippage Occurs . . . 12
  - 1.3.3 Three Drivers of Slippage . . . 12
- 1.4 Transaction Cost Models . . . 13
  - 1.4.1 Components of Transaction Costs . . . 13
  - 1.4.2 Flat or Fixed Cost Model . . . 14
  - 1.4.3 Linear Cost Model . . . 14
  - 1.4.4 Quadratic Cost Model . . . 14
  - 1.4.5 Using Historical Execution Costs . . . 15
- 1.5 Backtesting Framework . . . 15
  - 1.5.1 Complete Workflow . . . 15
  - 1.5.2 Common Backtesting Pitfalls . . . 16
- 1.6 Libraries . . . 17
  - 1.6.1 Algorithmic Trading Libraries . . . 17
  - 1.6.2 Pyfolio . . . 18
  - 1.6.3 Zipline . . . 24
  - 1.6.4 Backtesting.py . . . 33
  - 1.6.5 Putting It Together . . . 41

2 Stochastic Processes **42**
- 2.1 Deterministic vs. Stochastic Models . . . 42
  - 2.1.1 Deterministic Models . . . 42
  - 2.1.2 Stochastic Models . . . 42
  - 2.1.3 Comparison . . . 42
- 2.2 Mathematical Foundations of Stochastic Processes . . . 43

2.2.1 Probability Spaces . . . 43
2.2.2 Measurable Functions . . . 44
2.2.3 Formal Definition . . . 44
2.2.4 Sample Paths . . . 45
2.2.5 Stationarity . . . 45
2.2.6 Stationary and Independent Increments . . . 46
2.2.7 Filtrations: Information Through Time . . . 47
2.2.8 Adapted Processes . . . 47
2.2.9 Moment-Generating Functions . . . 48
2.2.10 Characteristic Functions . . . 48
2.2.11 Gaussian Random Variables . . . 48
2.2.12 Random Vectors and Covariance Matrices . . . 49
2.2.13 Gaussian Random Vectors . . . 49
2.2.14 Martingales . . . 50
2.3 Concept Map . . . 51

3 Markov Chains **52**
3.1 Motivation . . . 52
3.1.1 Why the Independence Assumption Can Fail . . . 52
3.2 Core Structure and Probability Propagation . . . 53
3.2.1 Definition and Markov Property . . . 53
3.2.2 Transition Matrices . . . 53
3.2.3 State-Distribution Vectors . . . 54
3.2.4 Chapman–Kolmogorov Equations . . . 55
3.2.5 Twelve-Month Loan Transition Probabilities . . . 55
3.3 State Classification and Long-Run Behaviour . . . 56
3.3.1 Accessibility, Communication, and Irreducibility . . . 56
3.3.2 Recurrence, Transience, and Absorption . . . 56
3.3.3 Periodicity . . . 56
3.3.4 Stationary Distributions and Ergodicity . . . 57
3.4 Model Design and Estimation . . . 57
3.4.1 Choosing States in Quantitative Finance . . . 57
3.4.2 Maximum Likelihood Estimation: General Idea . . . 58
3.4.3 Deriving the Markov-Chain MLE . . . 58
3.4.4 Loan-Portfolio MLE Example . . . 59
3.4.5 Assumptions Behind the Estimator . . . 59
3.4.6 Large-Sample Properties of the MLE . . . 60
3.5 Limitations and Extensions . . . 60

4 Random Walk **61**
4.1 Simple Symmetric Random Walk . . . 61
4.1.1 Definition and Rule . . . 61
4.1.2 Coin-Flip Analogy . . . 61
4.1.3 The Growing Tree of Possible Positions . . . 61
4.2 Counting Random-Walk Paths . . . 62
4.2.1 Paths Between Two Space–Time Points . . . 62
4.2.2 Total Number of Paths . . . 62
4.2.3 Number of Paths to a Fixed Endpoint . . . 63

4.2.4 A Useful Recursion . . . 63
4.2.5 Translation Invariance . . . 63
4.3 Return Probabilities . . . 64
4.4 The Reflection Principle . . . 64
4.4.1 Why the Reflection Principle Works . . . 65
4.4.2 Counting Paths That Avoid a Barrier . . . 65
4.5 Interpretation and Limitations in Finance . . . 66
4.6 Random Walk as the Discrete Precursor to Brownian Motion . . . 66

5 Derivatives **68**
5.1 Brownian Motion . . . 68
5.2 Modelling Stock Price . . . 68
5.3 Itô's Lemma and Girsanov's Theorem . . . 68
5.4 Factors Affecting Option Prices . . . 68
5.5 Black–Scholes . . . 68
5.6 Deriving the Black–Scholes Equation . . . 68
5.7 Options Pricing . . . 68
5.8 Binomial Model . . . 68
5.9 Option Greeks . . . 68

6 Machine Learning in Financial Markets **69**
6.1 Sentiment Analysis: Opinion Mining . . . 69
6.2 Research Papers . . . 69
6.3 Datasets for Self-Practice . . . 69
6.4 Time Series Analysis: Handling Volatility . . . 69
6.5 Sequence-to-Sequence Modelling in Finance . . . 69

Chapter 1

Algorithmic Trading

1.1 What is Backtesting?

1.1.1 Definition

Backtesting is the process of simulating a trading strategy on **historical market data** to estimate how it would have performed in the past. It is the main quantitative screening step before paper trading or committing real capital.

> **Core idea.** If this strategy had been used over the previous N years with the information available at each point in time, what returns, risks, and losses would it have produced?

1.1.2 Why Backtest?

Without a backtest, a trader has little evidence that a strategy is viable. A well-designed backtest provides the following benefits.

Benefit	Explanation
Risk-free evaluation	Tests a strategy on historical observations without risking actual capital.
Confidence building	Shows whether favourable results persist across time periods and market regimes.
Rejection or refinement	Reveals weak assumptions and parameters before money is committed.
Quantitative measurement	Turns a qualitative trading idea into measurable returns, risks, and costs.

1.1.3 The Backtesting Principle

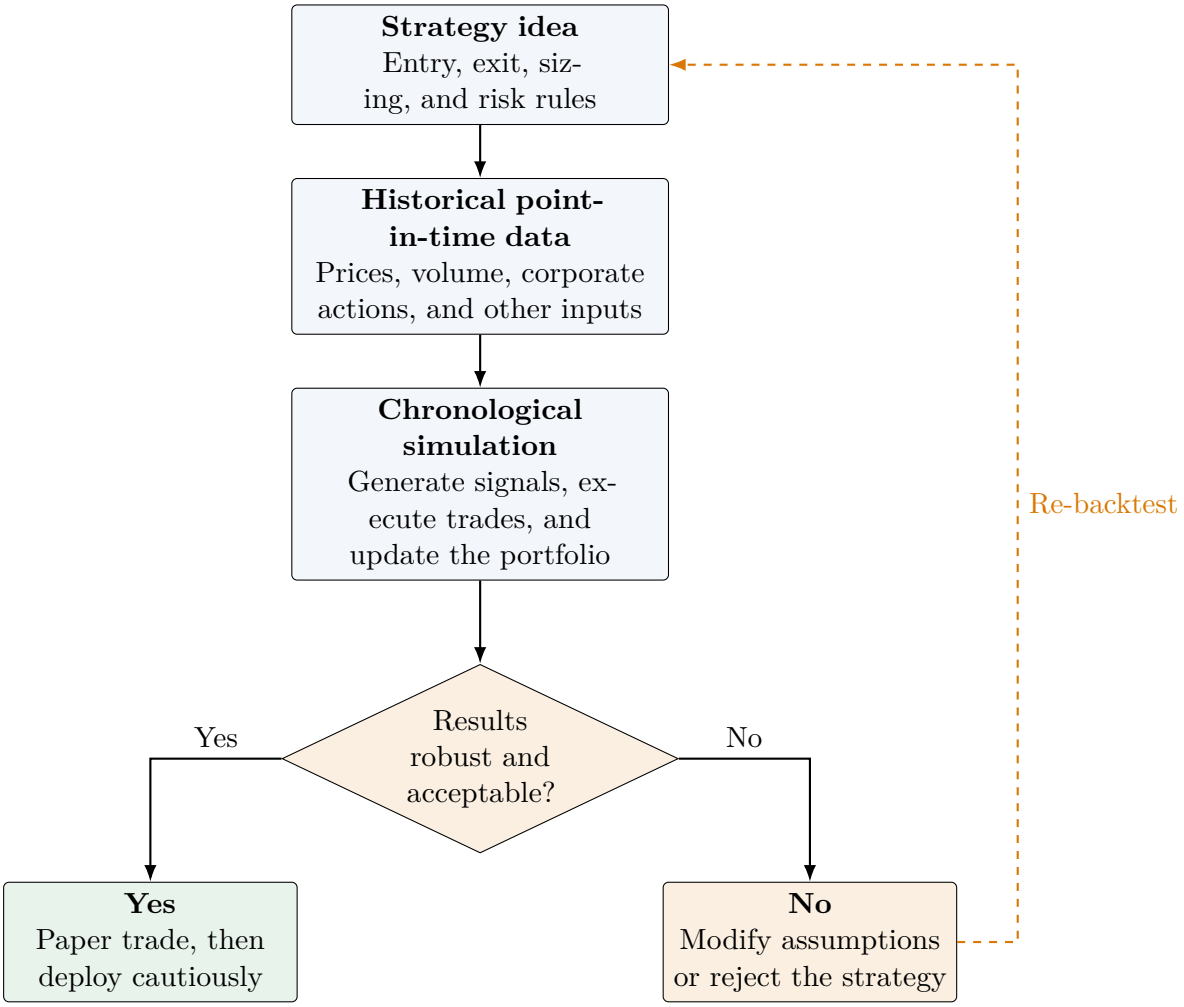


Figure 1.1: The basic backtesting and refinement cycle.

1.1.4 Requirements of a Good Backtest

1. **Quality historical data:** clean, complete, correctly adjusted, and free from survivorship bias.
2. **Realistic simulation:** transaction costs, bid–ask spreads, slippage, latency, liquidity, and market impact are represented.
3. **Appropriate time horizon:** the test covers several market regimes, including rising, falling, volatile, and sideways markets.
4. **Rigorous evaluation:** several return and risk measures are considered rather than raw profit alone.

Warning: overfitting. A strategy that looks exceptional on the data used to design it may merely have been tuned to historical noise. Reserve unseen data for out-of-sample testing and use walk-forward validation before drawing conclusions.

1.2 Performance Metrics for Backtesting a Trading Strategy

1.2.1 Overview

Performance metrics permit objective comparisons among strategies. **No single metric gives a complete assessment**; returns, risk, and risk-adjusted performance should be examined together.

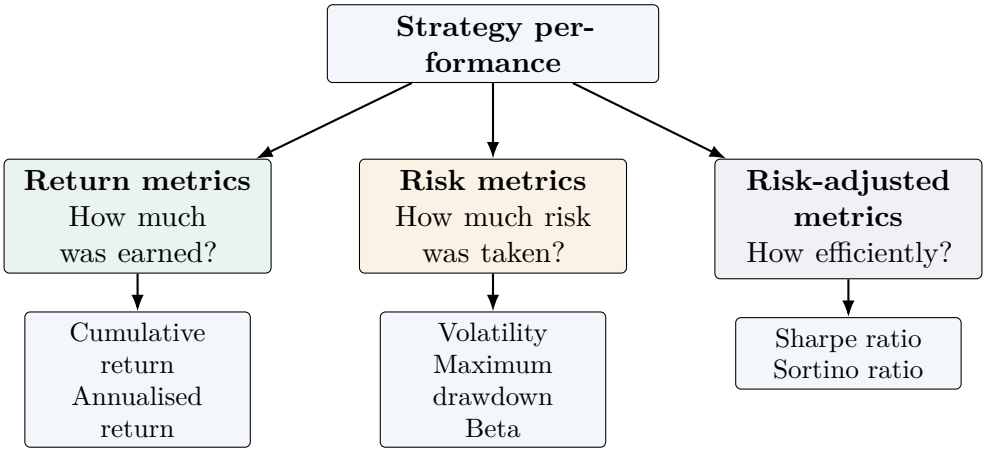


Figure 1.2: Three complementary views of strategy performance.

1.2.2 Cumulative Return (Absolute Return)

Definition. The total percentage gain or loss between the beginning and end of the backtest, irrespective of its duration:

$$\text{Cumulative return} = \frac{V_f - V_0}{V_0} \times 100\%,$$

where V_0 and V_f are the initial and final portfolio values.

Example.
If a portfolio grows from Rs.1,00,000 to Rs.1,45,000, then

$$\frac{1{,}45{,}000 - 1{,}00{,}000}{1{,}00{,}000} \times 100\% = \mathbf{45}\%.$$

Limitation. Cumulative return ignores time. A 45% return over ten years is not comparable with a 45% return earned in one year unless both are annualised.

1.2.3 Annualised Return

Definition. The annualised return, or compound annual growth rate (CAGR), is the constant yearly rate that produces the observed cumulative return over the same period:

$$\text{Annualised return} = (1 + \text{cumulative return})^{365/N} - 1,$$

where N is the number of calendar days in the backtest. If the duration is expressed as T years, the exponent is $1/T$.

Example.
A cumulative return of 45% over 547 days gives

$$(1.45)^{365/547} - 1 \approx (1.45)^{0.667} - 1 \approx \mathbf{28.1}\%.$$

Strategy	**Duration**	**Cumulative return**	**Annualised return**
A	3 years	60%	$(1.60)^{1/3} - 1 = 16.96\%$
B	1.5 years	45%	$(1.45)^{2/3} - 1 \approx 28.1\%$

Strategy B has the lower cumulative return but the higher annualised return because it earned its gain in half the time.

1.2.4 Annualised Volatility

Definition. Volatility measures the dispersion of periodic returns. For daily data it is commonly annualised as

$$\sigma_{\text{annual}} = \sigma_{\text{daily}}\sqrt{252},$$

where 252 is the conventional number of trading days in a year.

The square-root factor arises because independent return variances add over time: annual variance is $252\sigma^2_{\text{daily}}$, so annual standard deviation is $\sqrt{252}\sigma_{\text{daily}}$.

Example.
For a daily standard deviation of 1.3%,

$$0.013\sqrt{252} = 0.206 \approx \mathbf{20.6}\%.$$

Higher volatility means that realised returns are less stable. Thus, two strategies with equal annualised returns can have very different risk profiles.

1.2.5 Sharpe Ratio

Definition. The Sharpe ratio measures excess return per unit of total volatility:

$$\text{Sharpe ratio} = \frac{R_p - R_f}{\sigma_p},$$

where R_p is the portfolio return, R_f is the risk-free return, and σ_p is portfolio volatility. All three quantities must use the same time scale.

Sharpe ratio	**Typical interpretation**
< 0	The strategy earned less than the risk-free asset.
0 to 1	Positive but weak risk-adjusted performance.
1 to 2	Good risk-adjusted performance.
2 to 3	Very good risk-adjusted performance.
> 3	Exceptional in a backtest; verify carefully for overfitting and omitted risks.

Worked example.
For an annual strategy return of 18%, an annual risk-free rate of 5%, and annualised volatility of 12%,

$$\text{Sharpe ratio} = \frac{18\% - 5\%}{12\%} = \frac{13\%}{12\%} = \mathbf{1.08}.$$

Limitations.

- Upside and downside volatility are penalised equally.
- The measure can understate tail risk when returns are skewed or heavy-tailed.
- Results are sensitive to the sampling frequency, annualisation convention, and chosen risk-free rate.

1.2.6 Sortino Ratio

Definition. The Sortino ratio replaces total volatility with downside deviation, so only returns below a chosen target T are penalised:

$$\text{Sortino ratio} = \frac{R_p - R_f}{\sigma_{\text{down}}}, \qquad \sigma_{\text{down}} = \sqrt{\frac{1}{n}\sum_{i=1}^{n} \min(0, R_i - T)^2}.$$

The target may be zero, the risk-free rate, or another minimum acceptable return.

Example.

For periodic returns $5\%, -3\%, 8\%, -2\%, 4\%$ and $T = 0$,

$$\sigma_{\text{down}} = \sqrt{\frac{(-3)^2 + (-2)^2}{5}} = \sqrt{2.6} = 1.61\%.$$

If the annual return is 12%, the risk-free rate is 4%, and the downside deviation has been expressed on the same annual scale, then

$$\text{Sortino ratio} = \frac{12\% - 4\%}{1.61\%} = 4.97.$$

The Sortino ratio is especially useful for asymmetric strategies, but a high value should not be interpreted without examining rare tail losses.

1.2.7 Beta

Definition. Beta measures the sensitivity of a strategy to a market benchmark:

$$\beta = \frac{\text{Cov}(R_p, R_m)}{\text{Var}(R_m)},$$

where R_p and R_m are portfolio and market returns.

Beta	**Meaning**	**Example**
$\beta < 0$	Tends to move opposite to the market	Inverse or short strategies
$\beta = 0$	No linear market exposure	Market-neutral strategy
$0 < \beta < 1$	Moves with the market, but less strongly	Defensive portfolio
$\beta = 1$	Has the same market sensitivity as the benchmark	Index-tracking portfolio
$\beta > 1$	Amplifies market movements	Leveraged or aggressive portfolio

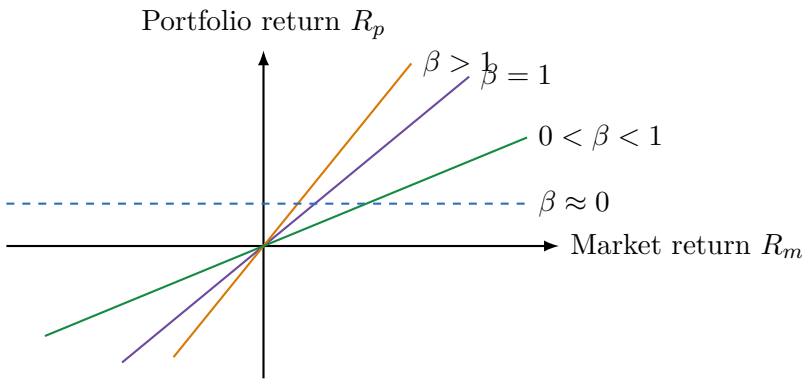


Figure 1.3: Beta is the slope of the portfolio–market return relationship.

A common single-factor decomposition is

$$R_p = \alpha + \beta R_m + \varepsilon,$$

where α is benchmark-independent performance and ε is unexplained variation. A high-return strategy with $\beta = 1.5$ may mostly represent leveraged market exposure rather than a distinct trading edge.

1.2.8 Maximum Drawdown

Definition. Maximum drawdown (MDD) is the largest peak-to-trough decline in portfolio value before a new peak is reached. Its positive magnitude is

$$\text{MDD} = \max_t \left(\frac{P_t - V_t}{P_t} \right) \times 100\%, \qquad P_t = \max_{s \le t} V_s.$$

Worked example.
Consider the portfolio path

$$\text{Rs. } 1{,}00{,}000 \to \text{Rs. } 1{,}50{,}000 \to \text{Rs. } 90{,}000 \to \text{Rs. } 1{,}20{,}000 \\ \to \text{Rs. } 70{,}000 \to \text{Rs. } 1{,}60{,}000.$$

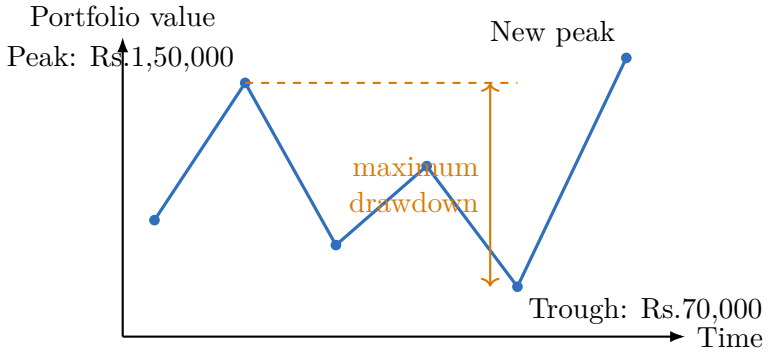


Figure 1.4: Maximum drawdown is measured from the preceding peak, not from the final value.

Thus,

$$\text{MDD} = \frac{1{,}50{,}000 - 70{,}000}{1{,}50{,}000} \times 100\% = \mathbf{53.33}\%.$$

The interim recovery to Rs.1,20,000 does not reset the peak because it does not exceed Rs.1,50,000.

A 50% loss requires a 100% subsequent gain merely to break even. MDD therefore gives an intuitive capital-preservation and psychological stress measure that volatility alone does not provide.

Two useful related measures are

$$\text{Calmar ratio} = \frac{\text{annualised return}}{|\text{MDD}|}, \qquad \text{Recovery factor} = \frac{\text{net profit}}{|\text{MDD}|}.$$

1.2.9 Summary of Performance Metrics

Metric	**Formula**	**Measures**	**Preferred direction**
Cumulative return	$(V_f - V_0)/V_0$	Total gain or loss	Higher
Annualised return	$(1 + CR)^{365/N} - 1$	Compounded yearly growth	Higher
Annualised volatility	$\sigma_d\sqrt{252}$	Dispersion of returns	Lower, generally
Sharpe ratio	$(R_p - R_f)/\sigma_p$	Excess return per unit of total risk	Higher
Sortino ratio	$(R_p - R_f)/\sigma_{\text{down}}$	Excess return per unit of downside risk	Higher
Beta	$\text{Cov}(R_p, R_m)/\text{Var}(R_m)$	Market sensitivity	Depends on objective
Maximum drawdown	$(P - T)/P$	Worst peak-to-trough decline	Lower

1.3 Slippage and Latency

1.3.1 What is Slippage?

Slippage is the difference between the price observed when a signal is generated and the price at which the order is actually filled:

$$\text{Slippage magnitude} = |\text{execution price} - \text{signal price}|\,.$$

For a buy order, execution above the signal price is adverse; for a sell order, execution below the signal price is adverse.

> **Example.** An algorithm generates a buy signal for HDFC Bank at Rs.1,650.00. The order is filled at Rs.1,651.25, producing adverse slippage of Rs.1.25 per share.

Slippage is a real transaction cost and can turn a small theoretical edge into a loss.

1.3.2 Why Slippage Occurs

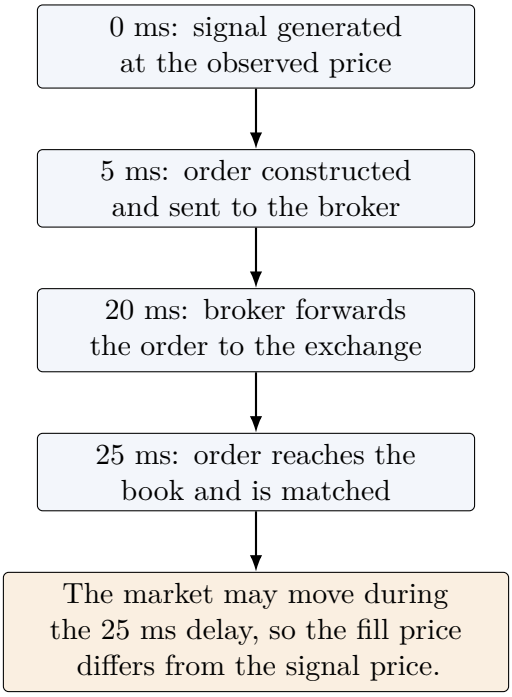


Figure 1.5: A simplified signal-to-execution latency chain.

1.3.3 Three Drivers of Slippage

Underlying Asset Volatility

The farther a price can move per unit of time, the larger the range of possible execution prices during a fixed delay.

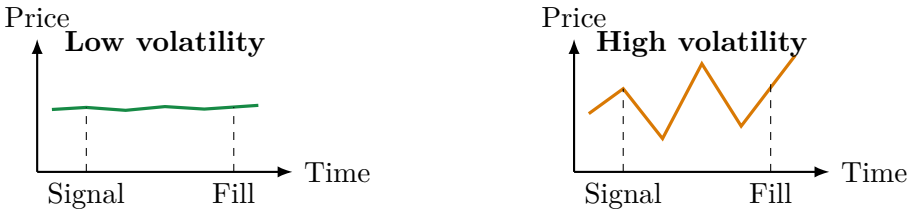


Figure 1.6: The same execution delay creates more slippage uncertainty in a volatile asset.

> **Rule of thumb.** Higher short-horizon volatility generally implies higher expected slippage, all else equal.

Latency

Latency is the delay between signal generation and execution. It includes:

- **algorithmic latency**: time used to calculate the signal, perform checks, and construct the order; and
- **communication and execution latency**: time used to send the order through the broker, reach the exchange, queue, and obtain a fill.

Hence,

$$\text{total latency} = \text{processing latency} + \text{communication and execution latency.}$$

Strategy type	**Sensitivity**	**Reason**
High-frequency trading	Extremely high	Signals may decay within microseconds or milliseconds.
Intraday algorithms	Moderate to high	Opportunities often last seconds or minutes.
Swing or position trading	Usually lower	Signals are intended to persist for days or weeks.

This is why latency-sensitive firms may colocate servers near exchange matching engines and use specialised networking or hardware.

Strategy Type

Strategy	**Typical impact**	**Reason**
Momentum	Often adverse	The strategy buys into rising prices or sells into falling prices.
Mean reversion	Can be less adverse, but is not guaranteed to be favourable	The price may continue moving before reverting.
Market making	Two-sided and complex	Limit-order fills, queue position, spread capture, and adverse selection all matter.

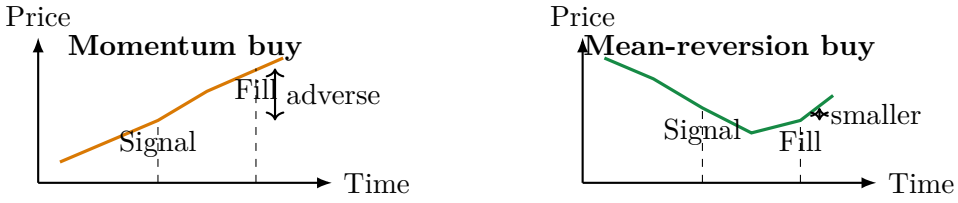


Figure 1.7: Illustrative, not guaranteed, slippage patterns for two strategy types.

1.4 Transaction Cost Models

1.4.1 Components of Transaction Costs

Transaction costs include every cost caused by executing and maintaining a trade. Omitting them can materially overstate backtested performance. Important components include:

- brokerage commissions;
- exchange fees, stamp duty, and taxes;
- the bid–ask spread;
- slippage and latency effects; and
- market impact, especially for orders large relative to available liquidity.

1.4.2 Flat or Fixed Cost Model

A fixed model assigns the same monetary cost to every trade:

$$\text{total transaction cost} = C_{\text{fixed}} N_{\text{trades}}.$$

For a fee of Rs.20 per trade and 500 trades,

$$\text{total cost} = \text{Rs. } 20 \times 500 = \text{Rs. } 10{,}000.$$

Advantages.

It is simple, computationally inexpensive, and can represent genuinely fixed per-order fees.

Disadvantages.

It ignores trade size, liquidity, spread, volatility, slippage, and market impact. It is therefore only a rough approximation for small and relatively uniform orders.

1.4.3 Linear Cost Model

A linear model makes cost proportional to traded notional V:

$$\text{cost} = cV.$$

For a 0.1% cost rate on a trade worth Rs.5,00,000,

$$\text{cost} = 0.001 \times \text{Rs. } 5{,}00{,}000 = \text{Rs. } 500.$$

A piecewise-linear specification may include a minimum charge, different brokerage tiers, or different spread assumptions across size ranges. Linear models are more realistic than fixed models for percentage fees, but they still miss nonlinear market impact.

1.4.4 Quadratic Cost Model

A simple nonlinear approximation is

$$\text{cost} = c_1 V + c_2 V^2,$$

or, in a purely quadratic specification, cost $= cV^2$. The quadratic term approximates the increasing marginal cost of consuming more levels of the order book.

For example, a purchase of 100 shares may be absorbed at the best ask, whereas a purchase of 10,000 shares may consume several price levels. The trader's own demand then raises the average execution price.

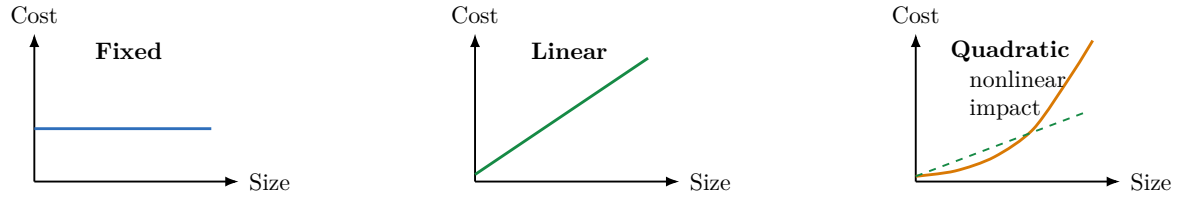


Figure 1.8: Fixed, linear, and quadratic transaction-cost approximations.

Advantages.

A calibrated nonlinear model can represent market impact more realistically when order size is material relative to liquidity.

Disadvantages.

The coefficients must be estimated for each asset, venue, order type, and market regime. A quadratic form is still an approximation; using it without calibration does not guarantee accuracy.

1.4.5 Using Historical Execution Costs

A more data-driven cost model can be constructed as follows:

1. collect fill prices, decision prices, order sizes, timestamps, and contemporaneous market conditions from executed orders;
2. estimate slippage as a function of order size, volatility, liquidity, spread, participation rate, and latency; and
3. apply the fitted model to simulated orders in the backtester.

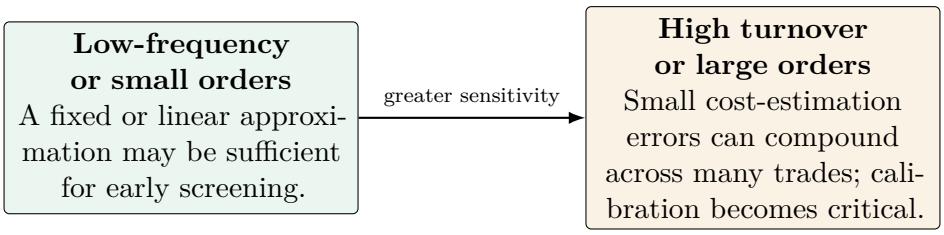


Figure 1.9: The required cost-model detail grows with turnover and market impact.

For a high-turnover strategy, even a few basis points of omitted cost per trade can consume the entire forecast edge.

1.5 Backtesting Framework

1.5.1 Complete Workflow

A rigorous backtest follows a staged process.

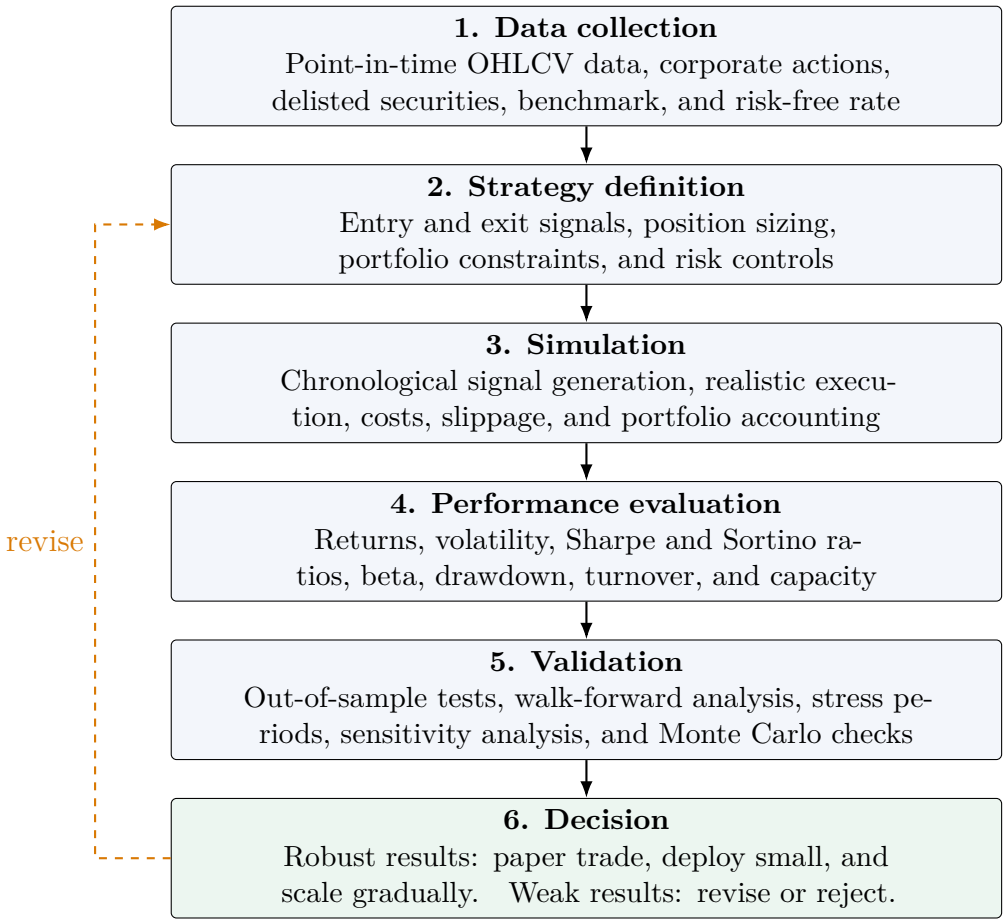


Figure 1.10: A complete backtesting and validation workflow.

1.5.2 Common Backtesting Pitfalls

Look-Ahead Bias

Look-ahead bias occurs when the strategy uses information that was not available at the simulated decision time. For example, today's closing price cannot generate a trade executed earlier during today's session.

Mitigation: enforce strict timestamp ordering and delay execution until after every required input would genuinely have been observable.

Survivorship Bias

Survivorship bias occurs when a historical universe contains only securities that still exist today. Failed, delisted, or removed securities are omitted, which can make past results look artificially strong.

Mitigation: use point-in-time index constituents and include all securities that were investable on each historical date.

Overfitting or Curve Fitting

Overfitting occurs when parameters are tuned so aggressively that the strategy captures noise specific to the development sample.

Mitigation: use a small number of economically justified parameters, reserve out-of-sample data, perform walk-forward validation, and test parameter sensitivity.

Ignoring Transaction Costs

Ignoring commissions, taxes, spreads, slippage, and market impact can turn an unprofitable strategy into an apparently profitable one. For example, a 0.1% round-trip cost repeated 1,000 times equals 100% of the repeatedly traded notional, although its exact effect on portfolio return depends on position size and turnover.

Mitigation: calibrate costs to the asset, venue, order type, liquidity, trade size, and strategy frequency.

Latency Blindness

Latency blindness assumes an instantaneous fill at the signal price. In practice, both the market and the available queue can change before an order arrives.

Mitigation: delay fills realistically and model order type, queue position, partial fills, and slippage at the relevant time scale.

1.6 Libraries

1.6.1 Algorithmic Trading Libraries

> **Note on sources:** These notes are compiled from the official documentation and tutorials for each library (linked below), since these are the canonical, most reliable, and most current references. I was not able to transcribe the linked YouTube playlists directly (no video/audio access), but the concepts, functions, and worked examples they'd typically cover for these libraries are the same ones documented in the official tutorials, which are used here as the primary source. If you want specific clarifications that came from the videos and aren't reflected here, let me know what topic/timestamp and I can look into it further.

Sources used:

- Pyfolio: https://quantopian.github.io/pyfolio/ (Overview + all 6 tutorial notebooks)
- Zipline (maintained fork *zipline-reloaded*): https://zipline.ml4trading.io/ (Tutorial + full API reference)
- Zipline-trader (Alpaca-focused live-trading fork): https://zipline-trader.readthedocs.io/en/latest/beginner-tutorial.html
- Backtesting.py: https://kernc.github.io/backtesting.py/doc/backtesting/ (API docs, `backtesting.backtesting`, `backtesting.lib`) + Quick Start User Guide

---

Quick Orientation: Which Library Does What?

Library	Role	Scope	Best for

Zipline	Event-driven **backtesting simulator** (runs a strategy bar-by-bar)	Multi-asset, portfolio-level	Writing and *simulating* a trading algorithm against historical data, including realistic order execution, slippage, commissions
Pyfolio	**Performance & risk analytics** (post-hoc analysis)	Works on the *output* (returns/positions/transactions) of a backtest (e.g. from Zipline) or even a single asset's returns	Generating professional "tear sheets" — the statistics and plots used to *evaluate* a strategy after it has been run
Backtesting.py	Lightweight, single-asset **backtesting + optimization** framework	One tradeable asset at a time; not built for multi-asset portfolio rebalancing	Quickly prototyping, backtesting, and optimizing indicator/signal-based strategies with an intuitive, minimal API and interactive plots

In short: you might **write and run** a strategy in **Zipline**, then **analyze its performance** with **Pyfolio** — or, for simpler single-asset strategy iteration, use **Backtesting.py** end-to-end (it has its own built-in stats and plotting, so it doesn't need Pyfolio).

1.6.2 Pyfolio

What It Is

Pyfolio is a Python library for **performance and risk analysis of financial portfolios**, originally developed by Quantopian Inc. It integrates well with **Zipline** (Zipline produces the returns/positions/transactions data that Pyfolio analyzes), but it can just as easily analyze the returns of a single asset compared to a benchmark, or returns from any other source.

Installation:

```
pip install pyfolio
```

(Bayesian tear sheets additionally require `pip install theano` and `pip install pymc3`.)

The Core Concept: Tear Sheets

At the heart of Pyfolio is the **tear sheet** — a collection of individual plots and statistics tables that together give a comprehensive picture of a strategy's performance. Rather than compute metrics one by one, you call one function and get a full battery of analysis at once.

The main tear-sheet-generating functions are:

Function	Purpose
`pf.create_returns_tear_sheet(returns, ...)`	Core statistics + plots based on a returns series alone (works even for a single stock)
`pf.create_full_tear_sheet(returns, positions=None, transactions=None, ...)`	The "everything" tear sheet: returns stats + positions + transactions + (optionally) round trips + slippage sweeps
`pf.create_position_tear_sheet(returns, positions, sector_mappings=None)`	Analysis of holdings over time (top positions, sector allocation)
`pf.create_round_trip_tear_sheet(returns, positions, transactions, sector_mappings=None)`	Analysis of individual "round trip" trades (win rate, profit factor, holding durations, etc.)

Required Input Data

Pyfolio tear sheets are built from up to three pandas objects:

- **`returns`** — a `pd.Series` of daily percentage returns, datetime-indexed.
- **`positions`** — a `pd.DataFrame` of daily dollar values held per symbol (plus a `'cash'` column), datetime-indexed.
- **`transactions`** — a `pd.DataFrame` of individual trades executed (datetime-indexed), used for transaction-cost/slippage and round-trip analysis.

Getting a Single Stock's Returns

```
import pyfolio as pf
stock_rets = pf.utils.get_symbol_rets('FB')
pf.create_returns_tear_sheet(stock_rets, live_start_date='2015-12-1')
```

`live_start_date` splits the analysis into **in-sample** (before that date) and **out-of-sample** (after) — useful for checking whether a strategy's live performance matches its backtested performance.

Getting Data From a Zipline Backtest

If you ran a strategy in Zipline and saved the results (`results.pickle`), Pyfolio can extract the three required objects directly:

```
import pandas as pd
import pyfolio as pf

results = pd.read_pickle('results.pickle')
returns, positions, transactions = pf.utils.extract_rets_pos_txn_from_zipline(
    results)
```

The Statistics Table (What Each Metric Means)

Every `create_*_tear_sheet` call prints a summary table like this (columns split into **All / In-sample / Out-of-sample** if `live_start_date` is given):

Metric	Meaning
Annual return	Compounded annualized return
Cumulative returns	Total return over the whole period
Annual volatility	Annualized standard deviation of daily returns
Sharpe ratio	Risk-adjusted return: excess return per unit of total volatility
Calmar ratio	Annual return divided by max drawdown — rewards strategies with shallow drawdowns
Stability	R^2 of a linear fit to the log cumulative returns curve — how smooth/consistent the equity growth is
Max drawdown	Largest peak-to-trough decline in cumulative returns
Omega ratio	Probability-weighted ratio of gains vs. losses relative to a threshold
Sortino ratio	Like Sharpe, but only penalizes *downside* volatility (not upside swings)
Skew	Asymmetry of the returns distribution (positive skew = more extreme gains than losses)
Kurtosis	"Fat-tailedness" of the returns distribution (high kurtosis = more extreme outlier days)
Tail ratio	Ratio of the size of the right tail (gains) to the left tail (losses), e.g. at the 95th vs. 5th percentile
Daily value at risk	Estimated daily loss threshold not expected to be exceeded except in the worst X% of cases
Gross leverage	Total absolute exposure (long + short) relative to portfolio value
Daily turnover	How much of the portfolio is traded each day, as a fraction of portfolio value
Alpha	Return not explained by the benchmark (skill/edge)
Beta	Sensitivity of the strategy's returns to the benchmark's returns

Below the summary table, tear sheets also show a **"Worst drawdown periods"** table listing each major drawdown's net decline (%), peak date, valley date, recovery date, and duration (in days) — this makes it easy to see not just *how deep* losses got but *how long* the strategy stayed underwater.

Plots Generated

Depending on which tear sheet function is called, Pyfolio can generate:

- Cumulative returns plot (linear and log scale), vs. benchmark
- Returns plot (per-period)
- Rolling 6-month volatility
- Rolling 6-month Sharpe ratio
- Top 5 drawdown periods (highlighted on the cumulative returns curve)
- Drawdown "underwater" plot (depth of drawdown over time)
- Monthly returns heatmap (%) and annual returns bar chart
- Distribution of monthly returns (histogram)
- Return quantiles box plot
- Stress event table (mean/min/max returns during historical stress periods like the 2008 Lehman crisis, Aug '07, Mar '08, Sept '08, the 2009 recovery, etc.) — lets you see how the strategy would have behaved during known crisis windows
- Top 10 long/short positions of all time, and all positions ever held (with sector breakdown, if `sector_mappings` is supplied)

Worked Example: Full Zipline Algorithm → Pyfolio Tear Sheet

This is the canonical workflow: run a Zipline algorithm, save its output, then feed that output into Pyfolio.

```
import pyfolio as pf
import zipline
%load_ext zipline
!zipline ingest   # download/prepare pricing data bundle first
```

```
%%zipline --start 2004-1-1 --end 2010-1-1 -o results.pickle

# An "On-Line Portfolio Moving Average Reversion" (OLMAR) strategy
import numpy as np
from zipline.finance import commission, slippage

STOCKS = ['AMD', 'CERN', 'COST', 'DELL', 'GPS', 'INTC', 'MMM']

def initialize(algo, eps=1, window_length=5):
    algo.stocks = STOCKS
    algo.sids = [algo.symbol(s) for s in algo.stocks]
    algo.m = len(algo.stocks)
    algo.b_t = np.ones(algo.m) / algo.m   # start with an equal-weighted
   portfolio
    algo.eps = eps                         # reversion sensitivity parameter
    algo.window_length = window_length
    algo.set_commission(commission.PerShare(cost=0))
    algo.set_slippage(slippage.FixedSlippage(spread=0))

def handle_data(algo, data):
    # Compute each asset's price relative to its recent moving average
    mavgs = data.history(algo.sids, 'price', algo.window_length, '1d').mean()
    x_tilde = np.array([mavgs[sid] / data.current(sid, "price") for sid in
   algo.sids])
```

```
    # ... mean-reversion portfolio update logic (project onto the simplex
    # of valid portfolio weights) ...
    rebalance_portfolio(algo, data, new_weights)

def rebalance_portfolio(algo, data, desired_port):
    for i, sid in enumerate(algo.sids):
        algo.order_target_percent(sid, desired_port[i])
```

Then extract the results and feed them straight to Pyfolio:

```
import pandas as pd
results = pd.read_pickle('results.pickle')
returns, positions, transactions = pf.utils.extract_rets_pos_txn_from_zipline(
    results)

# One plot at a time...
pf.plot_drawdown_periods(returns, top=5).set_xlabel('Date')

# ...or the full tear sheet, including round-trip trade analysis
pf.create_full_tear_sheet(returns, positions=positions, transactions=
    transactions,
                          live_start_date='2009-10-22', round_trips=True)
```

If you'd rather not reveal which specific symbols your strategy traded (e.g. when sharing a tear sheet publicly), pass `hide_positions=True`:

```
pf.create_full_tear_sheet(returns, positions=positions, transactions=
    transactions,
                          live_start_date='2009-10-22', hide_positions=True)
```

Sector Analysis

To break down PnL and position exposure by **sector** instead of just by symbol, pass a `sector_mappings` dictionary (symbol → sector name):

```
sect_map = {'COST': 'Consumer Goods', 'INTC': 'Technology', 'CERN': '
    Healthcare',
            'GPS': 'Technology', 'MMM': 'Construction', 'DELL': 'Technology',
    'AMD': 'Technology'}

pf.create_position_tear_sheet(returns, positions, sector_mappings=sect_map)
# or, for round trips:
pf.create_round_trip_tear_sheet(returns, positions, transactions,
    sector_mappings=sect_map)
```

This adds a "Profitability (PnL / PnL total) per name" table aggregated at the **sector** level in addition to the per-symbol table, which is useful for spotting concentration risk (e.g. "is my strategy's profit really coming from one sector?").

Round-Trip Trade Analysis

A **round trip trade** begins when a new long or short position is opened, and ends when it is completely or partially closed. Round-trip analysis exists to answer: *is this strategy*

repeatedly profitable, or did a few lucky trades carry the whole equity curve?

How it works under the hood: Pyfolio reconstructs your full portfolio from the `transactions` you provide. When a new trade occurs, it checks whether shares of that symbol are already held from an earlier purchase; if so, it computes the PnL, return, and duration of that "round trip." At the very end of the data, any still-open positions are synthetically closed out (using the last available price) so that their PnL is captured too.

```
pf.create_round_trip_tear_sheet(returns, positions, transactions,
    sector_mappings=sect_map)
```

This produces four groups of statistics, each broken down into **All trades / Short trades / Long trades**:

Stat group	Key fields	Meaning
Summary stats	Total # round trips, Percent profitable, Winning/Losing/Even round trips	Basic win-rate bookkeeping
PnL stats	Total profit, Gross profit, Gross loss, Profit factor, Avg./Largest winning & losing trade	Profit factor = gross profit / |gross loss|; a value > 1 means the strategy makes more from winners than it loses from losers
Duration stats	Avg./Median/Longest/Shortest duration	How long, on average, capital is tied up in a trade
Return stats	Avg./Median returns (all / winning / losing), largest winning/losing trade (%)	Same PnL picture but expressed as % return per round trip rather than $

There's also a **per-symbol breakdown** of all the above (so you can see, e.g., that one ticker in your universe was actually a net drag on performance), and a **"Profitability per name"** bar chart.

Under the hood, manually:

```
rts = pf.round_trips.extract_round_trips(
    transactions, portfolio_value=positions.sum(axis='columns') / (returns +
    1))
pf.round_trips.print_round_trip_stats(rts)
```

`rts` is itself a DataFrame with one row per round trip: `close_dt`, `long` (bool), `open_dt`, `pnl`, `rt_returns`, `symbol`, `duration`, `returns`.

Slippage Sensitivity Analysis

Slippage is the difference between the price you expected to trade at and the price you actually got — it eats into real-world returns in a way backtests often underestimate. Pyfolio's transactions tear sheet can generate **"slippage sweep"** plots showing how the strategy's equity curve would look under a range of additional per-dollar slippage assumptions.

```
pf.create_full_tear_sheet(returns, positions, transactions, slippage=5)
```

Here, `slippage` is specified in **basis points** (1% = 100 bps). Passing `slippage=5` applies 5 bps of *additional* slippage on top of whatever is already baked into the returns you passed in, and shows the resulting performance for the main tear sheet plots.

Important nuance: the slippage sweep plots themselves always show a *range* of slippage scenarios (e.g. 0, 1, 2, 5, 10 bps) for comparison — the `slippage=N` argument only affects the *main* returns/plots elsewhere in the tear sheet, not the sweep itself. If your `returns` already reflect realistic slippage (e.g. from a proper Zipline backtest with a slippage model), pass `slippage=0` so you don't double-count, while still getting the sweep plot for comparison purposes.

Bayesian Tear Sheet

Pyfolio also offers a **Bayesian** analysis (requires PyMC3 + Theano) that models strategy returns probabilistically rather than with simple point-estimate statistics — e.g. giving credible intervals for the Sharpe ratio rather than a single number, and modeling the probability distribution of future drawdowns. This is more advanced/experimental and is best explored via the linked example notebook rather than summarized here, since it's essentially a from-scratch probabilistic model rather than a fixed formula.

1.6.3 Zipline

What It Is and Current Status

Zipline is an **open-source, event-driven algorithmic trading simulator** written in Python, originally built by Quantopian as the engine behind their backtesting and live-trading platform. When Quantopian shut down in late 2020, its original documentation domain (`zipline.io`) expired.

Important: the documentation used here is for **`zipline-reloaded`**, the actively maintained community fork (maintained by Stefan Jansen, author of *Machine Learning for Algorithmic Trading*) — this is the version you should actually install today, via:

```
pip install zipline-reloaded
# or
conda install -c conda-forge zipline-reloaded
```

There is also a separate fork, **`zipline-trader`**, aimed specifically at connecting Zipline to the Alpaca brokerage for **live trading**, not just backtesting — its beginner tutorial content is functionally identical to the core Zipline tutorial (same `initialize`/`handle_data` API), but it adds bundle ingestion via Alpaca, a YAML config file, and a "Going Live" mode.

Key stated benefits of Zipline:

- **Realistic**: models slippage, transaction costs, and order delays.
- **Stream-based**: processes each time-bar event individually, which helps avoid look-ahead bias.
- **Batteries included**: common transforms (e.g. moving averages) and risk stats (e.g. Sharpe) are available out of the box.
- **PyData integration**: input/output are Pandas DataFrames.

The Core Algorithm Structure

Every Zipline algorithm is built around **two required functions**:

```
def initialize(context):
    ...  # called once, before the backtest starts

def handle_data(context, data):
    ...  # called once per event (trading bar)
```

- **`initialize(context)`** runs once at the very start. `context` is a persistent namespace object — anything you store on it (`context.i = 0`, `context.asset = ...`) is available in every later call.
- **`handle_data(context, data)`** runs once per bar (once per day, for daily data; once per minute, for minute data). `data` is a `BarData` object giving you access to current and historical price/volume info for your universe.

Two additional optional functions:

- **`before_trading_start(context, data)`** — runs once per day, before the market opens (good place to run/refresh a Pipeline).
- **`analyze(context, perf)`** — runs once, automatically, after the whole backtest finishes; `perf` is the full performance DataFrame, useful for custom plotting.

Minimal Example (`buyapple.py`)

```
from zipline.api import order, record, symbol

def initialize(context):
    pass

def handle_data(context, data):
    order(symbol('AAPL'), 10)  # buy 10 shares of AAPL every bar
    record(AAPL=data.current(symbol('AAPL'), 'price'))
```

- `order()` takes an asset object and a signed integer amount — positive to buy/cover, negative to sell/short.
- `record()` saves a named value on every bar; after the run finishes, these appear as columns in the performance DataFrame (letting you plot e.g. your own custom indicator alongside portfolio value).

The `BarData` Object (`data`)

Passed into `handle_data`/`before_trading_start`, this is your window into current and historical prices.

`data.current(assets, fields)`

Returns the *current* bar's value(s) for one or more assets/fields.

- Valid `fields`: `"price"`, `"open"`, `"high"`, `"low"`, `"close"`, `"volume"`, `"last_traded"`.
- `"price"` = last known close, forward-filled if no trade happened this bar, and split/dividend-adjusted.

- `"open"/"high"/"low"/"close"` = NaN if there was no trade this bar.
- `"volume"` = 0 if no trade this bar.
- Return type depends on inputs: scalar (1 asset, 1 field) → `pd.Series` (1 asset+many fields, or many assets+1 field) → `pd.DataFrame` (many assets+many fields).

`data.history(assets, fields, bar_count, frequency)`

Returns a **trailing window** of historical data — this is how you get access to "yesterday's" or "last N days'" prices, adjusted for splits/dividends/mergers as of *now* (so no look-ahead bias creeps in via unadjusted historical prices).

- `frequency`: `'1d'` for daily bars, `'1m'` for minute bars (needs minute-resolution data ingested).
- `bar_count`: how many historical bars to return.
- Shape of the return value follows the same "scalar / Series / DataFrame" logic as `.current()`, with the extra wrinkle that multi-asset + multi-field requests return a `MultiIndex` DataFrame.

`data.can_trade(assets)` / `data.is_stale(assets)`

- `can_trade()`: True only if the asset is alive, its exchange is open right now, and there's a known last price for it — use this before ordering to avoid errors on delisted/inactive assets.
- `is_stale()`: True if the asset is alive but simply hasn't traded in the current bar (helps distinguish "no recent data" from "doesn't exist").

Dual Moving Average Example (Using `history`)

This is the canonical "beyond the basics" tutorial example, and it introduces the idea of **precomputing indicators from a rolling history window**:

```
from zipline.api import order_target, record, symbol

def initialize(context):
    context.i = 0
    context.asset = symbol('AAPL')

def handle_data(context, data):
    context.i += 1
    if context.i < 300:
        return  # skip until we have a full 300-bar window

    short_mavg = data.history(context.asset, 'price', bar_count=100, frequency
    ="1d").mean()
    long_mavg  = data.history(context.asset, 'price', bar_count=300, frequency
    ="1d").mean()

    if short_mavg > long_mavg:
        order_target(context.asset, 100)    # go/stay long 100 shares
    elif short_mavg < long_mavg:
        order_target(context.asset, 0)      # flatten the position

    record(AAPL=data.current(context.asset, 'price'),
```

```
            short_mavg=short_mavg, long_mavg=long_mavg)

def analyze(context, perf):
    import matplotlib.pyplot as plt
    fig = plt.figure()
    ax1 = fig.add_subplot(211); perf.portfolio_value.plot(ax=ax1)
    ax2 = fig.add_subplot(212); perf['AAPL'].plot(ax=ax2)
    perf[['short_mavg', 'long_mavg']].plot(ax=ax2)
    plt.show()
```

Logic: this is a classic momentum "golden cross / death cross" strategy — go long when the short-term average crosses above the long-term average (recent prices trending up relative to the longer trend), and exit when it crosses back below.

A subtlety worth remembering: `order_target()` (like all the "target" order functions) does **not** account for orders you've already placed but which haven't filled yet. Calling `order_target(asset, 10)` twice in a row (before either fills) results in **20 shares total**, not 10 — because Zipline doesn't yet know the first order will succeed when it processes the second.

Running an Algorithm: Three Interfaces

A. Command Line

```
zipline ingest -b quandl      # first: download/prepare a data bundle
zipline run -f my_algo.py --start 2016-1-1 --end 2018-1-1 -o results.pickle --
    no-benchmark
```

Key `zipline run` flags:

Flag	Purpose
`-f/-algofile`	Path to your algorithm's `.py` file
`-b/-bundle`	Which ingested data bundle to use (default `quandl`)
`-s/-start`, `-e/-end`	Backtest date range
`-data-frequency`	`daily` (default) or `minute`
`-capital-base`	Starting cash (default 10,000,000)
`-o/-output`	Where to save the resulting performance DataFrame (pickle format)
`-benchmark-symbol` / `-benchmark-sid` / `-benchmark-file`	What to use as the benchmark for alpha/beta calculations
`-no-benchmark`	Skip a benchmark entirely (alpha/beta won't be computed)
`-trading-calendar`	Which market calendar to simulate against (default NYSE)

Zipline first calls `initialize()`, then streams historical bars through `handle_data()` day by day (or minute by minute). After `handle_data()` returns each bar, Zipline looks at any orders placed and tries to fill them against that bar's volume — applying the configured commission and slippage models, so the true cost is more than just `price` × `shares`.

B. Jupyter Notebook Magic

```
%load_ext zipline
```

```
%%zipline --start 2016-1-1 --end 2018-1-1 -o perf_variable
from zipline.api import symbol, order, record

def initialize(context):
    pass

def handle_data(context, data):
    order(symbol('AAPL'), 10)
    record(AAPL=data.current(symbol('AAPL'), 'price'))
```

Same CLI-style flags apply, but the algorithm lives inline in the notebook cell, and `-o name` creates a Python variable (rather than writing a file) holding the resulting DataFrame.

C. `run_algorithm()` — for Running Like a Normal Script/IDE

```
from zipline import run_algorithm
import pandas as pd
import pandas_datareader.data as web

start = pd.Timestamp('2014'); end = pd.Timestamp('2018')
sp500 = web.DataReader('SP500', 'fred', start, end).SP500
benchmark_returns = sp500.pct_change()

result = run_algorithm(
    start=start.tz_localize('UTC'), end=end.tz_localize('UTC'),
    initialize=initialize, handle_data=handle_data,
    capital_base=100000, benchmark_returns=benchmark_returns,
    bundle='quandl', data_frequency='daily')
```

This is the most flexible option since it's just a Python function call — good for running inside an IDE, a script, or a testing/CI pipeline.

The Performance DataFrame

Whichever way you run it, Zipline returns/saves a `pd.DataFrame` with **one row per trading day (or minute)**. Key columns include:

Column	Meaning
`portfolio_value`	Total account value (cash + holdings) at that point
`returns`	Period return
`algorithm_period_return` / `benchmark_period_return`	Cumulative return so far, strategy vs. benchmark
`alpha`, `beta`, `sharpe`, `sortino`	Rolling risk/performance stats, computed automatically
`algo_volatility`, `benchmark_volatility`	Rolling annualized volatility

`max_drawdown`	Running max drawdown to date
`positions`	List of currently held positions (dicts, with sid, amount, cost basis, last sale price, etc.)
`transactions`	List of any fills that happened this bar
`orders`	List of any orders created/updated this bar
`ending_cash`, `ending_value`, `ending_exposure`	Cash and market value of holdings at end of bar
`longs_count` / `shorts_count`, `long_value` / `short_value`	Breakdown of long vs. short exposure
`gross_leverage` / `net_leverage`	Leverage measures
Any name you passed to `record()`	Your custom tracked values (e.g. `AAPL`, `short_mavg`)

Placing Orders

Function	Semantics
`order(asset, amount, limit_price=None, stop_price=None, style=None)`	Order a **fixed number of shares** (+ = buy/cover, − = sell/short)
`order_value(asset, value, ...)`	Order enough shares to transact a **fixed dollar value**. Equivalent to `order(asset, value / current_price)`
`order_percent(asset, percent, ...)`	Order enough shares to allocate a given **% of current portfolio value** to the asset
`order_target(asset, target, ...)`	Order the difference needed so the position reaches a **target share count**
`order_target_value(asset, target, ...)`	Same, but target is a **dollar value** (for Futures, this becomes target *exposure*, since futures have no "value" per se)
`order_target_percent(asset, target, ...)`	Same, but target is a **% of portfolio value**

Critical caveat repeated in the docs for all four "target" functions: none of them account for your own still-open orders. Calling the same `order_target*` twice before the first fills will double the effect, since each call only looks at your *current filled* position, not your pending orders.

Execution Styles (how/when an order fills)

- `MarketOrder` (default) — filled at the current/next available market price.
- `LimitOrder(limit_price)` — filled only at `limit_price` or better.
- `StopOrder(stop_price)` — becomes a market order once price crosses `stop_price` (in the direction that would normally hurt you, i.e. this is a protective/breakout trigger).

- `StopLimitOrder(limit_price, stop_price)` — combination: becomes a *limit* order once the stop price is triggered.

You can pass `limit_price=` / `stop_price=` directly to `order()` as shorthand instead of constructing a style object explicitly (but not both a `style=` and `limit_price=`/`stop_price=` at once).

Managing Orders

- `get_order(order_id)` — look up a specific order.
- `get_open_orders(asset=None)` — all open orders (dict of asset → list, or a list if `asset` given).
- `cancel_order(order_param)` — cancel by ID or Order object.
- **Cancel policies** — `set_cancel_policy(cancel_policy)`, with built-ins `EODCancel()` (cancel all open orders at end of day; minute-frequency sims only) and `NeverCancel()` (default; orders are Good-Til-Canceled).

Looking Up Assets

- `symbol(symbol_str, country_code=None)` — look up a single Equity by ticker (as of the "symbol lookup date," since tickers can be reused by different companies over time).
- `symbols(*args)` — look up several tickers at once, returns a list.
- `future_symbol(symbol)` — look up a Future contract by its symbol.
- `sid(sid)` — look up any Asset by its permanent integer ID (doesn't change over time, unlike ticker symbols).
- `set_symbol_lookup_date(dt)` — change the reference date used to resolve ticker symbols to assets.

Trading Controls (Safety Rails)

These raise a `TradingControlException` if violated — designed to prevent a bug (or, on a live account, a fat-finger) from doing serious damage:

- `set_do_not_order_list(restricted_list)` — assets that can never be ordered.
- `set_long_only()` — disallow short positions entirely.
- `set_max_leverage(max_leverage)` — cap on account leverage.
- `set_max_order_count(max_count)` — cap on number of orders per day.
- `set_max_order_size(asset=None, max_shares=None, max_notional=None)` — cap the size of any single order.
- `set_max_position_size(asset=None, max_shares=None, max_notional=None)` — cap the size of any resulting position (note: splits/dividends or price moves can still push you slightly over after the fact).

Commission and Slippage Models

These model the two main sources of "backtest looks better than reality" error.

Commission (`set_commission(us_equities=..., us_futures=...)`), can only be set inside `initialize()`:

Model	Formula

`commission.PerShare(cost=0.001, min_trade_cost=0.0)`	Flat $ per share traded, with an optional minimum per trade. **This is Zipline's default for equities.**
`commission.PerTrade(cost=0.0)`	Flat $ per trade regardless of size (multi-fill orders charge the full amount on the first fill)
`commission.PerDollar(cost=0.0015)`	$ cost proportional to dollar value transacted (default 0.15% of value)

You can also subclass `CommissionModel` and implement `calculate(order, transaction)` for custom logic.

Slippage (`set_slippage(us_equities=..., us_futures=...)`):

Model	Formula / Behavior
`slippage.FixedSlippage(spread=0.0)`	Buys fill at `close + spread/2`; sells fill at `close - spread/2`. No volume/liquidity limit — will always fill in full as soon as *any* trading occurs, even if your order is bigger than the historical volume (unrealistic for large orders, but simple).
`slippage.VolumeShareSlippage(volume_limit=0.025, price_impact=0.1)`	Models slippage as a function of how much of the bar's volume your order represents. Buys fill at `price * (1 + price_impact * volume_share**2)`; sells at `price * (1 - price_impact * volume_share**2)`, where `volume_share` is the fraction of the bar's volume your order fills (capped at `volume_limit`, e.g. 2.5% of the bar's volume by default). This is more realistic since larger orders (relative to available liquidity) get worse fills.

You can also subclass `SlippageModel` and implement `process_order(data, order)` to build fully custom fill logic (it can even raise `LiquidityExceeded` to signal "no more of this order fills this bar").

Scheduling Recurring Logic

Rather than checking "is it a certain day/time?" manually inside `handle_data` every bar, you can register a function to run automatically on a schedule:

```
from zipline.api import schedule_function, date_rules, time_rules

schedule_function(my_rebalance_func,
                  date_rule=date_rules.week_start(),
                  time_rule=time_rules.market_open(minutes=30))
```

- **date_rules**: `every_day()`, `month_start(days_offset=0)`, `month_end(days_offset=0)`, `week_start(days_offset=0)`, `week_end(days_offset=0)`.
- **time_rules**: `every_minute`, `market_open(offset=None, hours=None, minutes=None)`, `market_close(offset=None, hours=None, minutes=None)` — default trigger time is 1 minute after open / 1 minute before close if no offset given.

This is the idiomatic way to implement e.g. "rebalance my portfolio on the first trading day of each month" without manual date-checking logic.

Benchmark, Recording, and Misc

- `set_benchmark(benchmark_asset)` — set which asset to use as the return benchmark (for alpha/beta); dividends on the benchmark are automatically reinvested in the calculation.
- `record(**kwargs)` — track custom named values per bar (shows up as columns in the performance DataFrame and in `analyze()`).
- `get_environment(field='platform')` — introspect the run itself: `'arena'` (backtest vs. live), `'data_frequency'`, `'start'`/`'end'`, `'capital_base'`, `'platform'`, or `'*'` for everything as a dict.
- `fetch_csv(url, ...)` — pull in an arbitrary external CSV (e.g. a sentiment index or alternative dataset) and make it queryable through `data`, keyed by date (and optionally by symbol).

The Pipeline API (Cross-Sectional Computations)

A **Pipeline** lets you efficiently compute the same calculation (a "Factor," "Filter," or "Classifier") across your **entire tradeable universe** at once, once per day — much faster and more memory-efficient than looping over each asset yourself inside `handle_data`. This is the mechanism used for things like "rank all 8,000 US stocks by 20-day momentum" or "select all stocks whose 10-day average volume exceeds $1M."

Registering and reading a pipeline:

```
from zipline.api import attach_pipeline, pipeline_output

def initialize(context):
    pipe = Pipeline(columns={...}, screen=my_filter)
    attach_pipeline(pipe, 'my_pipeline')

def before_trading_start(context, data):
    context.pipeline_data = pipeline_output('my_pipeline')
```

- `attach_pipeline(pipeline, name, chunks=None, eager=True)` — registers a pipeline to run once per trading day.
- `pipeline_output(name)` — fetches that day's already-computed results as a DataFrame (rows = assets that passed the `screen`, columns = the pipeline's declared `columns`).

Core Pipeline building blocks:

- **`Filter`** — a Boolean-output term (e.g. "10-day VWAP $<$ $20"). Filters are commonly built via comparison operators on Factors (`<`, `<=`, `>`, `>=`, `!=`, `.eq()`), and

combined with `&` (and), `|` (or), `~` (not). A Filter set as a Pipeline's `screen` drops non-passing asset/date rows from the output entirely.

- **`Factor`** — a numerical (or date) output term; the most common Pipeline term. Factors support arithmetic (`+`, `-`, `*`, `/`), and useful transformation methods:
  - `.rank(method='ordinal', ascending=True, mask=None, groupby=None)` — cross-sectional rank each day
  - `.zscore(mask=None, groupby=None)` — standardize `(row - mean) / std` each day
  - `.demean(mask=None, groupby=None)` — subtract the row mean
  - `.winsorize(min_percentile, max_percentile, ...)` — clip extreme values to a percentile band (limits outlier influence without fully discarding them)
  - `.quantiles(bins, mask=None)` — bucket each day's cross-section into `bins` quantile groups
  - `.pearsonr(target, correlation_length, mask=None)` / `.spearmanr(...)` / `.linear_regression(target, regression_length, mask=None)` — rolling cross-sectional statistical relationships against a target series
- **`CustomFactor`** — the base class for writing your **own** Factor. You override `compute(self, today, assets, out, *inputs)`, where `inputs` are NumPy arrays shaped `(window_length, num_assets)` for each declared input column, and you write your result into `out`. Example:

```
class TenDayRange(CustomFactor):
    inputs = [USEquityPricing.high, USEquityPricing.low]
    window_length = 10

    def compute(self, today, assets, out, highs, lows):
        out[:] = np.nanmax(highs, axis=0) - np.nanmin(lows, axis=0)
```

Why this matters: the Pipeline engine handles all the tricky bookkeeping (aligning windows across assets with different listing dates, forward-filling NaNs for not-yet-listed stocks, batching computation efficiently) so your `compute()` function can just focus on the math.

The Blotter

A **blotter** is the component that actually turns your placed orders into transactions each bar — it books the trade, applying the active slippage and commission models. Zipline's default is `SimulationBlotter`, and you generally don't need to touch this directly unless you're building a custom execution model; it's exposed mainly for extensibility (e.g. to log every order/fill for custom analysis, or to implement custom order-batching logic via `batch_order()`).

1.6.4 Backtesting.py

What It Is

Backtesting.py is a small, fast, dependency-light (Pandas/NumPy/Bokeh) Python framework purpose-built for backtesting **single-asset** strategies driven by technical indicators/signals — it deliberately does **not** try to support multi-asset portfolio rebalancing

or stock-picking across a universe (that's Zipline's job). Its selling point is a tiny, memorable API, fast vectorized execution, and beautiful **interactive** (Bokeh-based) plots and heatmaps, all built in — no need to pair it with something like Pyfolio.

Installation:

```
pip install backtesting
```

Data Format

You bring your own OHLC(V) data as a `pandas.DataFrame` with columns `'Open'`, `'High'`, `'Low'`, `'Close'`, and optionally `'Volume'`. If you're missing some columns (e.g. you only have close prices), you can fake them: `df['Open'] = df['High'] = df['Low'] = df['Close']`. The index should ideally be a **datetime index**; otherwise a simple monotonic range index works too. Extra columns (e.g. a sentiment score) are allowed and remain accessible to your strategy.

The package ships a small sample dataset for experimentation:

```
from backtesting.test import GOOG   # daily OHLCV for Google, 2004--2013
```

Defining a Strategy

A strategy subclasses `Strategy` and overrides two methods:

```
from backtesting import Strategy
from backtesting.lib import crossover
import pandas as pd

def SMA(values, n):
    return pd.Series(values).rolling(n).mean()

class SmaCross(Strategy):
    n1 = 10   # class variables => automatically optimizable parameters
    n2 = 20

    def init(self):
        # Precompute indicators in a vectorized way, over the FULL dataset
        self.sma1 = self.I(SMA, self.data.Close, self.n1)
        self.sma2 = self.I(SMA, self.data.Close, self.n2)

    def next(self):
        # Called once per bar; only "sees" data up to the current bar
        if crossover(self.sma1, self.sma2):
            self.position.close()
            self.buy()
        elif crossover(self.sma2, self.sma1):
            self.position.close()
            self.sell()
```

- `init()` runs once, before simulation starts. This is where you declare/precompute indicators — you have access to the *entire* dataset here (for efficient vectorized computation), but nothing you do here can leak into `next()`'s trading decisions in

a look-ahead sense, because...

- **`next()`** is called once per bar as the simulation steps forward in time. Inside `next()`, `self.data` and any `self.I()`-declared indicator arrays are **truncated to only what would have been known so far** — i.e. `array[-1]` is always "now," `array[-2]` is "one bar ago," etc. This is the mechanism that prevents look-ahead bias.
- **Important limitation:** Backtesting.py cannot simulate trading *within* a candle. New orders are filled at the **next bar's open** (or the *current* bar's close, if you set `trade_on_close=True`). If you need intra-candle decisions (day-trading logic), you need more granular (e.g. hourly/minute) source data instead.
- **`self.I(func, *args, name=None, plot=True, overlay=None, color=None, scatter=False, **kwargs)`** wraps an indicator-computing function so that (a) it's automatically plotted with a sensible legend/label, and (b) it participates in the same "gradually revealed" windowing as `self.data`. Rolling indicators typically front-pad with NaN for their warm-up period — **the backtest doesn't actually begin trading until the first bar where *all* declared indicators are non-NaN** (e.g. bar 201 if you use a 200-bar moving average), which can shift results if you change indicator lengths.

Why use `crossover()` instead of manual comparisons?

```
from backtesting.lib import crossover
crossover(self.sma1, self.sma2)   # True exactly on the bar sma1 crosses above
    sma2
```

This is equivalent to (but much more readable than) writing:

```
self.sma1[-2] < self.sma2[-2] and self.sma1[-1] > self.sma2[-1]
```

Running a Backtest

```
from backtesting import Backtest

bt = Backtest(GOOG, SmaCross, cash=10_000, commission=.002)
stats = bt.run()
```

`Backtest(data, strategy, ...)` Constructor Parameters

Parameter	Meaning
`cash`	Starting capital (default 10,000)
`spread`	Constant bid-ask spread, as a fraction of price (e.g. `0.0002` for a 0.2‰ forex spread)

`commission`	Commission rate. A single float (e.g. `0.01` = 1%, applied at **both** entry and exit), a tuple `(fixed, relative)` (e.g. `(100, .01)` = min $100 + 1%), or a callable `func(order_size, price) -> float` for fully custom commission logic. Negative values model a market-maker rebate.
`margin`	Required margin ratio for leveraged trading, e.g. `0.02` for 50:1 leverage (1/leverage)
`trade_on_close`	If `True`, market orders fill at the *current* bar's close instead of the *next* bar's open
`hedging`	If `True`, long and short positions can coexist; if `False` (default), opposite orders close existing trades first, FIFO-style
`exclusive_orders`	If `True`, every new order automatically closes out the prior trade first — enforces "at most one open trade (long or short) at a time," letting you skip manual `self.position.close()` calls
`finalize_trades`	If `True`, any trade still open at the very end of the backtest is force-closed on the last bar so it's included in the statistics (otherwise it's excluded as "unrealized")

There's also `FractionalBacktest` (in `backtesting.lib`) for assets like Bitcoin where you need to trade fractional units — it works by rescaling price/volume by a `fractional_unit` (default: 1 satoshi) so the underlying whole-share engine can still be used.

The `run()` Output — Statistics Explained

`bt.run()` returns a `pd.Series` of results. Example output (annotated):

Stat	Meaning
`Start`, `End`, `Duration`	Backtest date range and span
`Exposure Time [%]`	% of the time the strategy held *any* position (long or short)
`Equity Final [$]` / `Equity Peak [$]`	Final and highest account value reached
`Return [%]`	Total return over the period
`Buy & Hold Return [%]`	What you'd have made just holding the asset the whole time — the natural benchmark
`Return (Ann.) [%]`	Annualized return
`Volatility (Ann.) [%]`	Annualized volatility of the equity curve
`CAGR [%]`	Compound annual growth rate

`Sharpe Ratio` / `Sortino Ratio` / `Calmar Ratio`	Same definitions as in the Pyfolio section above
`Alpha [%]` / `Beta`	Performance/sensitivity relative to buy & hold
`Max. Drawdown [%]` / `Avg. Drawdown [%]`	Depth of worst / typical drawdown
`Max. Drawdown Duration` / `Avg. Drawdown Duration`	How long the worst / a typical drawdown lasted
`# Trades`	Total number of round-trip trades taken
`Win Rate [%]`	% of trades that were profitable
`Best Trade [%]` / `Worst Trade [%]` / `Avg. Trade [%]`	Individual trade return extremes and average
`Max. Trade Duration` / `Avg. Trade Duration`	How long trades are typically held
`Profit Factor`	Gross profit / gross loss (same meaning as in Pyfolio)
`Expectancy [%]`	Average expected % return per trade
`SQN`	Van Tharp's **System Quality Number** — a measure combining trade count, average PnL, and PnL standard deviation, used as **Backtesting.py's default metric to maximize during optimization**
`Kelly Criterion`	The theoretically optimal fraction of capital to risk per trade, per the Kelly formula
`_strategy`	The Strategy instance (and its parameter values) used for this run
`_equity_curve`	Full DataFrame: `Equity`, `DrawdownPct`, `DrawdownDuration` per bar
`_trades`	Full DataFrame: one row per closed trade, with entry/exit bar, price, PnL, return %, duration, and (helpfully) the value of every declared indicator at entry and exit

Trading Inside `next()`: Orders, Positions, and Trades

Placing Orders

```
self.buy(size=.9999, limit=None, stop=None, sl=None, tp=None, tag=None)
self.sell(size=.9999, limit=None, stop=None, sl=None, tp=None, tag=None)
```

- **`size`**: if between 0 and 1, interpreted as a **fraction of currently available liquidity** (cash + unrealized P/L − used margin) to commit; if ≥ 1, an **absolute number of units**.
- **`limit`/`stop`**: set these to place limit/stop(-limit) orders instead of a plain market order.

- **`sl`/`tp`**: stop-loss / take-profit price — automatically creates a linked contingent order that closes the resulting Trade once that price is touched. These are also **writable properties on an existing `Trade`** (`trade.sl = new_price`), letting you implement trailing stops by updating them inside `next()`.
- **`tag`**: an arbitrary marker you can attach to track a specific order/trade through to the results (e.g. to tell apart different entry signals in `_trades`).
- **Caution:** `self.sell(size=.1)` does **not** automatically close an existing `self.buy(size=.1)` position, *unless* `exclusive_orders=True`, or price/spread/commission are all zero and happen to match exactly. To deliberately exit a position, use `Position.close()` or `Trade.close()` instead of placing an opposite order and hoping it nets out.
- Unless `trade_on_close=True`, market orders fill on the **next** bar's open; limit/stop orders fill whenever their trigger condition is met.

`Order` (pending, not yet filled)

Properties: `is_long`, `is_short`, `is_contingent` (True for SL/TP bracket orders auto-placed against an open Trade), `limit`, `stop`, `size`, `sl`, `tp`, `tag`. Method: `cancel()`.

`Position` (`self.position`) — your current net holding

Truthy/falsy in boolean context (`if self.position:`). Properties: `is_long`, `is_short`, `size` (negative if short), `pl` (cash P/L), `pl_pct` (% P/L). Method: `close(portion=1.0)` — closes `portion` of every active trade.

`Trade` (`self.trades` = open, `self.closed_trades` = settled)

Properties: `size`, `entry_bar`/`exit_bar`, `entry_price`/`exit_price`, `entry_time`/`exit_time`, `is_long`/`is_short`, `pl`/`pl_pct` (commissions only reflected once closed), `value` (size × price), `sl`/`tp` (writable — reassign to modify, `None` to cancel), `tag` (inherited from the order that opened it). Method: `close(portion=1.0)`.

Helper Functions (`backtesting.lib`)

Function	Purpose
`crossover(series1, series2)`	True if `series1` just crossed **above** `series2` on this bar
`cross(series1, series2)`	True if the two series crossed in **either** direction
`barssince(condition, default=inf)`	How many bars since `condition` was last True
`quantile(series, quantile=None)`	If `quantile=None`, returns the *rank* (0–1) of the series' last value among its history; if given a value in (0,1), returns the *value* at that quantile

`resample_apply(rule, func, series, *args, agg=None, **kwargs)`	Apply an indicator function to `series` after resampling it to a different (usually coarser) timeframe — e.g. compute a **daily** SMA(10) inside an **hourly** strategy, then forward-fill it back down to hourly resolution, all with correct handling to avoid look-ahead bias (`label='right'` on the resample)
`plot_heatmaps(heatmap, agg='max', ...)`	Plot a grid of 2D heatmaps from an `optimize(..., return_heatmap=True)` result, one per pair of parameters, aggregating any remaining dimensions with `agg`
`compute_stats(*, stats, data, trades=None, risk_free_rate=0.0)`	Recompute the summary statistics from a (possibly filtered) subset of trades — e.g. compute stats using **only the long trades** from a run: `compute_stats(stats=stats, trades=stats._trades[stats._trades.Size > 0], data=GOOG)`
`random_ohlc_data(example_data, frac=1.0, random_state=None)`	Generates synthetic OHLC data with similar descriptive statistics to real `example_data` — useful for Monte Carlo robustness/stress-testing of a strategy against data it wasn't fit to

Composable Base Strategies

- **`SignalStrategy`** — subclass this, call `super().init()`, then `self.set_signal(entry_size, exit_portion=None)` with precomputed entry/exit signal arrays (positive = long signal, negative = short signal). This effectively lets you write a **vectorized** ("all signals precomputed up front") style strategy instead of manually issuing orders inside `next()`.
- **`TrailingStrategy`** — adds an automatic **ATR-based trailing stop-loss**. Call `set_trailing_sl(n_atr=6)` to trail the price by `n_atr` average-true-range units, or `set_trailing_pct(pct=0.05)` to trail by a percentage (internally converted to ATR units), and optionally `set_atr_periods(periods=100)` to change the ATR lookback.
- **`MultiBacktest`** — runs the *same* strategy across **multiple instruments in parallel** (e.g. `MultiBacktest([EURUSD, BTCUSD], SmaCross)`), returning a DataFrame of stats per instrument — useful for checking whether a strategy is robust across different assets, not overfit to one.

Optimization

```
stats = bt.optimize(
    n1=range(5, 30, 5),
    n2=range(10, 70, 5),
    maximize='Equity Final [$]',
    constraint=lambda p: p.n1 < p.n2)
```

- Any keyword whose name matches a Strategy class variable becomes a **parameter sweep**; pass a list/range of candidate values.
- `maximize`: either the name of a column from the `run()` stats (e.g. `'Equity Final [$]'`, `'Sharpe Ratio'`), or a custom function `f(stats) -> float` to maximize. Default: Van Tharp's **SQN**.
- `method`: `"grid"` (default) — exhaustive (or, with `max_tries`, randomized) search over the full parameter grid; or `"sambo"` — model-based ("SAMBO") optimization that intelligently searches for a near-optimal combination in at most `max_tries` evaluations (default 200), much cheaper than grid search for large parameter spaces.
- `constraint`: a function over a parameter combination that returns `True` if that combination is admissible to test (e.g. rejecting `n1 >= n2`, which would be a meaningless "moving average" crossover).
- `return_heatmap=True`: additionally returns a `pd.Series` with a MultiIndex over every admissible parameter combination and its resulting score — feed this into `plot_heatmaps()` to visualize the whole parameter landscape (not just the single best point), which helps you judge whether the "best" result is a robust plateau or an overfit spike.
- `random_state`: set for reproducible optimization runs.
- After optimizing, `stats._strategy` gives you the winning **Strategy instance**, e.g. `<Strategy SmaCross(n1=10,n2=15)>`.

Overfitting warning (explicit in the docs): optimizing on a single in-sample period can make performance look far better than it will be live — always be skeptical of a heavily-tuned single backtest result, and prefer checking whether nearby parameter combinations (visible in the heatmap) perform similarly well, rather than only trusting the single "best" point.

Plotting

```
bt.plot(plot_volume=True, plot_pl=True, plot_drawdown=False, plot_trades=True,
    ...)
```

Produces an interactive **Bokeh** HTML chart (opened in your browser by default) with:

- The OHLC candlestick chart (optionally with larger-timeframe candles "superimposed," e.g. daily candles over hourly data)
- All indicators declared via `self.I()`, automatically placed either overlaying price or in their own subplot
- Equity curve (`plot_equity`) or cumulative return curve (`plot_return`) — mutually complementary views of the same thing (equity = return + 100%)
- Profit/loss markers (`plot_pl`), trade volume (`plot_volume`), a separate drawdown-depth chart (`plot_drawdown`)
- "Tractor beam" hash-marks between each trade's entry and exit (`plot_trades`)
- `resample=True` automatically downsamples very long OHLC series so Bokeh doesn't choke on plotting >10,000 candles, while still aggregating equity/drawdown/trade data sensibly for the coarser view

Worked Example Summary (Google Stock, SMA Crossover)

Running the `SmaCross` strategy (10/20-day SMA) on ~9 years of Google daily data with `cash=10_000, commission=.002`:

- Untuned (`n1=10, n2=20`): ~**463%** return vs. **607%** buy & hold, Sharpe ≈ 0.60, Max Drawdown ≈ −34%.
- After optimizing `n1` over 5–30 and `n2` over 10–70 (constrained to `n1 < n2`), maximizing **Equity Final [$]**: the best combination (`n1=10, n2=15`) achieves ~**678%** return — beating buy & hold this time — but at a *worse* max drawdown (−44.5%) and lower Sharpe than you might hope, illustrating the classic **return-vs-risk (and overfitting) trade-off** that any optimization needs to be checked against.

1.6.5 Putting It Together

A realistic research pipeline combining these three tools might look like:

1. **Prototype and iterate quickly** on a single-asset idea in **Backtesting.py** — cheap to run, built-in optimizer, and good interactive plots for fast feedback.
2. Once the strategy needs a **realistic multi-asset universe, Pipeline-based stock selection, or more nuanced order/commission/slippage modeling**, port it to **Zipline**.
3. Use **Pyfolio** on the Zipline output (or on any returns series, really) to get a rigorous, standardized **tear sheet** — the in-sample/out-of-sample split, drawdown analysis, round-trip trade quality, and stress-period behavior are exactly the kind of due-diligence checks you'd want before trusting a strategy with real capital.

Chapter 2

Stochastic Processes

This chapter develops the probability and information structures used to describe random systems through time. It begins by distinguishing stochastic models from deterministic ones, then builds the formal language needed for random walks, Markov chains, Brownian motion, and derivative pricing.

2.1 Deterministic vs. Stochastic Models

2.1.1 Deterministic Models

Definition.
A deterministic model predicts outputs with **complete certainty** for a given set of inputs and parameters.

Characteristics.

- No randomness is involved.
- Repeating the process with the same inputs **always** yields the same result.

Examples.

- Calculating the position of the moon in its orbit.
- Computing the cost of electricity consumed from the units used and the price per unit.

2.1.2 Stochastic Models

Definition.
A stochastic model includes elements of **uncertainty and randomness** in its outputs.

Characteristics.

- The same inputs can produce **different outputs** because of underlying randomness.
- Outputs depend on **probability distributions**, not fixed formulas.

Examples.

- **Stock-price prediction:** affected by unpredictable news, market trends, and investor behaviour.
- **Crop-yield estimation:** influenced by weather, pests, and soil quality.

2.1.3 Comparison

Deterministic Models	Stochastic Models
Predictable outputs	Random outputs

No uncertainty	Includes uncertainty
The same inputs always produce the same result	The same inputs may produce different results
Suitable when all variables are known	Suitable for systems influenced by randomness

Why stock prices are stochastic, not deterministic. The cost of electricity is deterministic because it depends on fixed, known inputs: units consumed and the rate per unit. Stock prices, by contrast, are influenced by many unpredictable factors, so their evolution over time must be modelled as a **stochastic process**.

2.2 Mathematical Foundations of Stochastic Processes

Before defining a stochastic process formally, we need the mathematical structure on which random outcomes and their probabilities are defined.

2.2.1 Probability Spaces

A **probability space** is a triple

$$(\Omega, \mathcal{F}, \mathbb{P}),$$

with the following components.

1. **Sample space Ω.** This is the set of every possible outcome of the experiment.
2. **Sigma algebra $\mathcal{F}$.** This is a collection of subsets of Ω, called **events**, to which probabilities can be assigned.
3. **Probability measure $\mathbb{P}$.** This assigns a number $\mathbb{P}(A) \in [0, 1]$ to every event $A \in \mathcal{F}$.

Sigma-algebra requirements.

A collection $\mathcal{F} \subseteq \mathcal{P}(\Omega)$ is a sigma algebra if

1. $\Omega \in \mathcal{F}$;
2. if $A \in \mathcal{F}$, then $A^c \in \mathcal{F}$;
3. if $A_1, A_2, \ldots \in \mathcal{F}$, then $\bigcup_{k=1}^{\infty} A_k \in \mathcal{F}$.

These conditions also imply that $\varnothing \in \mathcal{F}$ and that countable intersections of measurable events belong to $\mathcal{F}$.

Probability-measure requirements.

For events in $\mathcal{F}$, a probability measure satisfies

$$\mathbb{P}(A) \geq 0, \qquad \mathbb{P}(\Omega) = 1,$$

and, for pairwise disjoint events $A_1, A_2, \ldots$,

$$\mathbb{P}\left(\bigcup_{k=1}^{\infty} A_k\right) = \sum_{k=1}^{\infty} \mathbb{P}(A_k).$$

Example: rolling a fair die.
The sample space is

$$\Omega = \{1, 2, 3, 4, 5, 6\}.$$

Define

$$A_{\text{odd}} = \{1, 3, 5\}, \qquad A_{\text{even}} = \{2, 4, 6\}.$$

If we record only whether the result is odd or even, the relevant sigma algebra is

$$\mathcal{F}_{\text{parity}} = \{\varnothing, \Omega, A_{\text{odd}}, A_{\text{even}}\}.$$

For a fair die,

$$\mathbb{P}(A) = \frac{|A|}{6}, \qquad A \subseteq \Omega,$$

so $\mathbb{P}(A_{\text{odd}}) = \mathbb{P}(A_{\text{even}}) = 1/2$. If every individual outcome must be distinguishable, one normally uses the full power set $\mathcal{F} = \mathcal{P}(\Omega)$.

2.2.2 Measurable Functions

Let $(E, \mathcal{E})$ and $(F, \mathcal{G})$ be measurable spaces. A function $f : E \to F$ is **measurable** if

$$B \in \mathcal{G} \quad \Longrightarrow \quad f^{-1}(B) = \{x \in E : f(x) \in B\} \in \mathcal{E}.$$

In words, every measurable event in the output space can be traced back to a measurable event in the input space. This condition is what makes expressions such as

$$\mathbb{P}(f \in B) = \mathbb{P}\Big(f^{-1}(B)\Big)$$

well defined.

For $\mathbb{R}^d$, the usual sigma algebra is the **Borel sigma algebra** $\mathcal{B}(\mathbb{R}^d)$, generated by all open subsets of $\mathbb{R}^d$.

2.2.3 Formal Definition

Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space, let T be an index set, and let $(E, \mathcal{E})$ be a measurable state space. A **stochastic process** is a family

$$X = (X_t)_{t \in T}$$

such that, for every fixed $t \in T$,

$$X_t : (\Omega, \mathcal{F}) \longrightarrow (E, \mathcal{E})$$

is a measurable random variable. Equivalently, it can be viewed as a map

$$X : T \times \Omega \longrightarrow E, \qquad (t, \omega) \longmapsto X_t(\omega).$$

Common choices are

$$T = \{0, 1, 2, \ldots\} \quad \text{or} \quad T = [0, \infty),$$

for discrete and continuous time respectively, and

$$E = \mathbb{R}^d, \qquad \mathcal{E} = \mathcal{B}(\mathbb{R}^d),$$

for a d-dimensional process.

Dimension	Process	Interpretation
One-dimensional	$X_t \in \mathbb{R}$	A stock price, interest rate, or one coordinate of a particle.
Two-dimensional	$X_t = (X_t^{(1)}, X_t^{(2)}) \in \mathbb{R}^2$	The position of a particle moving in a plane.
Three-dimensional	$X_t = (X_t^{(1)}, X_t^{(2)}, X_t^{(3)}) \in \mathbb{R}^3$	The position of a particle moving through physical space.

Continuous time does not imply continuous paths. Even when $T = [0, \infty)$, the function $t \mapsto X_t(\omega)$ may jump. A continuous-time process may therefore have continuous paths, jump paths, or more irregular behaviour.

Stock-price example.
If S_t denotes the price of a stock at time t, then each S_t is a random variable. Before the market evolves, S_t is unknown; after an outcome ω is realised, $S_t(\omega)$ is the observed price at time t.

2.2.4 Sample Paths

The two inputs t and ω have different roles:

- fixing t gives a random variable $\omega \mapsto X_t(\omega)$;
- fixing ω gives an ordinary function $t \mapsto X_t(\omega)$, called a **sample path** or **realisation**.

Different outcomes $\omega_1, \omega_2, \ldots$ generally generate different paths.

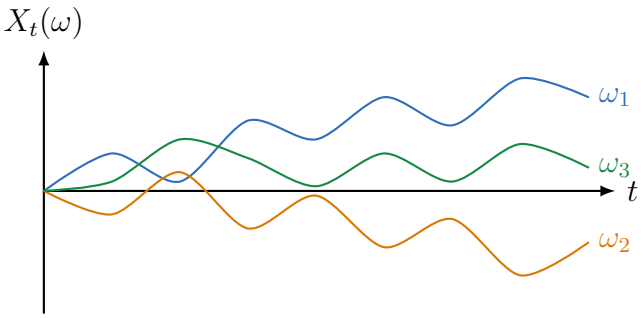


Figure 2.1: Different outcomes generate different sample paths of the same stochastic process.

2.2.5 Stationarity

A process $X = (X_t)$ is **strictly stationary** if shifting the time origin does not change any of its finite-dimensional distributions. Formally, for every n, every $t_1, \ldots, t_n$, and every admissible shift h,

$$(X_{t_1+h}, \ldots, X_{t_n+h}) \overset{d}{=} (X_{t_1}, \ldots, X_{t_n}).$$

The process therefore has the same probabilistic behaviour after a time shift.

A second-order process is **weakly stationary** if

$$\mathbb{E}[X_t] = m$$

is constant and

$$\mathrm{Cov}(X_s, X_t) = C(t-s)$$

depends only on the time lag.

> **Important distinction.** Stationarity does not say that the future is independent of the past. It says that the distributional rules are unchanged by shifting time. Dependence may still be present.

2.2.6 Stationary and Independent Increments

These are properties of changes in the process rather than of its absolute level.

Stationary increments.

The process has stationary increments if the distribution of

$$X_{t+h} - X_t$$

depends only on the interval length h, not on the starting time t. When $X_0 = 0$, this can be written

$$X_{t+h} - X_t \overset{d}{=} X_h.$$

Independent increments.

For every sequence

$$0 \le t_0 < t_1 < \cdots < t_n,$$

the increments

$$X_{t_1} - X_{t_0}, \quad X_{t_2} - X_{t_1}, \quad \ldots \quad , X_{t_n} - X_{t_{n-1}}$$

are independent random variables. Thus changes over non-overlapping time intervals do not influence one another.

For example, if $t_1 < t_2 \le t_3 < t_4$, independent increments imply that

$$X_{t_2} - X_{t_1} \quad \text{and} \quad X_{t_4} - X_{t_3}$$

are independent.

Property	Meaning
Stationary process	The joint distribution of the levels is unchanged by a common time shift.
Stationary increments	The distribution of a change depends only on the length of its interval.
Independent increments	Changes over disjoint time intervals are independent.

Standard Brownian motion has stationary and independent increments, but it is not itself stationary because $\mathrm{Var}(W_t) = t$ changes with time.

2.2.7 Filtrations: Information Through Time

A **filtration** is an increasing family of sub-sigma algebras

$$(\mathcal{F}_t)_{t\in T}$$

such that

$$s \le t \quad \Longrightarrow \quad \mathcal{F}_s \subseteq \mathcal{F}_t \subseteq \mathcal{F}.$$

The sigma algebra $\mathcal{F}_t$ represents all information available by time t. Information accumulates as time passes, so later sigma algebras are larger. The quadruple

$$(\Omega, \mathcal{F}, (\mathcal{F}_t)_{t\in T}, \mathbb{P})$$

is called a **filtered probability space**.

Three-coin-flip example.

Suppose a coin is flipped three times. Then

$$\Omega = \{HHH, HHT, HTH, HTT, THH, THT, TTH, TTT\}.$$

Let C_k denote the result of flip k, and define the natural filtration

$$\mathcal{F}_k = \sigma(C_1, \ldots, C_k).$$

Time	Information atoms	Interpretation	Events
0	Ω	No flip has been observed. Only impossible and certain events are distinguishable.	2
1	$A_H = \{HHH, HHT, HTH, HTT\}$, $A_T = \{THH, THT, TTH, TTT\}$	The first flip is known.	$2^2 = 4$
2	$A_{HH}, A_{HT}, A_{TH}, A_{TT}$, according to the first two flips	The first two flips are known.	$2^4 = 16$
3	The eight singleton outcomes	All three flips are known.	$2^8 = 256$

Each sigma algebra consists of every union of its information atoms. For example,

$$\mathcal{F}_0 = \{\varnothing, \Omega\}, \qquad \mathcal{F}_1 = \{\varnothing, \Omega, A_H, A_T\}.$$

If the realised sequence is HTH, then the information narrows from Ω at time 0, to A_H after the first flip, to A_{HT} after the second, and finally to the singleton $\{HTH\}$.

2.2.8 Adapted Processes

A process $X = (X_t)$ is **adapted** to the filtration $(\mathcal{F}_t)$ if

$$X_t \text{ is } \mathcal{F}_t\text{-measurable for every } t.$$

Adaptedness means that the value X_t can be determined from information available at time t; it does not use future information.

For a stock-price process S_t, adaptation means that events such as

$$\{S_t \le s\}$$

belong to $\mathcal{F}_t$. The market can observe the current and past prices, but not future prices.

2.2.9 Moment-Generating Functions

For a real random variable X, the **moment-generating function** (MGF) is

$$M_X(u) = \mathbb{E}\Big[e^{uX}\Big],$$

for those real u at which the expectation is finite. Using

$$e^{uX} = \sum_{k=0}^{\infty} \frac{u^k X^k}{k!},$$

and interchanging expectation and summation when justified gives

$$M_X(u) = \sum_{k=0}^{\infty} \frac{u^k}{k!} \mathbb{E}[X^k].$$

Hence, if the MGF exists on an open interval around zero,

$$M_X^{(k)}(0) = \mathbb{E}[X^k].$$

In particular,

$$\mathbb{E}[X] = M_X'(0), \qquad \mathbb{E}[X^2] = M_X''(0),$$

and

$$\mathrm{Var}(X) = M_X''(0) - \Big(M_X'(0)\Big)^2.$$

The MGF need not exist away from zero for every distribution. For a Gaussian random variable $X \sim \mathcal{N}(\mu, \sigma^2)$, however,

$$M_X(u) = \exp\!\left(\mu u + \frac{1}{2}\sigma^2 u^2\right).$$

2.2.10 Characteristic Functions

The **characteristic function** of a real random variable X is

$$\varphi_X(u) = \mathbb{E}\Big[e^{iuX}\Big], \qquad u \in \mathbb{R},$$

where $i^2 = -1$. Since $|e^{iuX}| = 1$, the characteristic function exists for every real-valued random variable and every real u, even when the MGF does not exist. It is the Fourier transform of the probability distribution of X and uniquely determines that distribution.

When the corresponding moments exist,

$$\varphi_X^{(k)}(0) = i^k \mathbb{E}[X^k].$$

2.2.11 Gaussian Random Variables

A Gaussian or normal random variable with mean μ and variance σ^2 is written

$$X \sim \mathcal{N}(\mu, \sigma^2)$$

and has density

$$f_X(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\!\left[-\frac{(x-\mu)^2}{2\sigma^2}\right].$$

Its characteristic function is

$$\boxed{\varphi_X(u) = \exp\!\left(i\mu u - \frac{1}{2}\sigma^2 u^2\right).}$$

For a standard normal variable $Z \sim \mathcal{N}(0,1)$, this becomes

$$\varphi_Z(u) = e^{-u^2/2}.$$

Derivation.
Substituting the Gaussian density into the definition gives

$$\varphi_X(u) = \frac{1}{\sqrt{2\pi}\sigma} \int_{-\infty}^{\infty} \exp\left(iux - \frac{(x-\mu)^2}{2\sigma^2} \right) dx.$$

With $x = \mu + \sigma z$,

$$\varphi_X(u) = e^{i\mu u} \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} \exp\left(-\frac{z^2}{2} + iu\sigma z \right) dz.$$

Completing the square,

$$-\frac{z^2}{2} + iu\sigma z = -\frac{(z - iu\sigma)^2}{2} - \frac{\sigma^2 u^2}{2}.$$

The remaining shifted Gaussian integral contributes 1 after normalization, which yields

$$\varphi_X(u) = e^{i\mu u} e^{-\sigma^2 u^2/2}.$$

2.2.12 Random Vectors and Covariance Matrices

An n-dimensional random vector is

$$\boldsymbol{X} = (X_1, \ldots, X_n)^{\mathsf{T}}.$$

Its mean is the vector

$$\boldsymbol{\mu} = \mathbb{E}[\boldsymbol{X}] = \begin{pmatrix} \mathbb{E}[X_1] \\ \vdots \\ \mathbb{E}[X_n] \end{pmatrix},$$

and its covariance matrix is

$$\boldsymbol{\Sigma} = \mathrm{Cov}(\boldsymbol{X}), \qquad \Sigma_{jk} = \mathrm{Cov}(X_j, X_k) = \mathbb{E}[(X_j - \mu_j)(X_k - \mu_k)] \,.$$

The matrix $\boldsymbol{\Sigma}$ is symmetric and positive semidefinite. Its diagonal entries are variances, while its off-diagonal entries measure linear co-movement between components.

The multivariate characteristic function is

$$\varphi_{\boldsymbol{X}}(\boldsymbol{u}) = \mathbb{E}\left[e^{i\boldsymbol{u}^{\mathsf{T}}\boldsymbol{X}} \right], \qquad \boldsymbol{u} \in \mathbb{R}^n.$$

2.2.13 Gaussian Random Vectors

A random vector $\boldsymbol{X}$ is **jointly Gaussian** if every linear combination

$$\boldsymbol{u}^{\mathsf{T}}\boldsymbol{X}$$

is a one-dimensional Gaussian random variable. If

$$\boldsymbol{X} \sim \mathcal{N}_n(\boldsymbol{\mu}, \boldsymbol{\Sigma}),$$

then

$$\boldsymbol{u}^{\mathsf{T}}\boldsymbol{X} \sim \mathcal{N}\left(\boldsymbol{u}^{\mathsf{T}}\boldsymbol{\mu}, \boldsymbol{u}^{\mathsf{T}}\boldsymbol{\Sigma}\boldsymbol{u} \right),$$

and

$$\boxed{\varphi_{\boldsymbol{X}}(\boldsymbol{u}) = \exp\Big(i\boldsymbol{u}^{\mathsf{T}}\boldsymbol{\mu} - \frac{1}{2}\boldsymbol{u}^{\mathsf{T}}\boldsymbol{\Sigma}\boldsymbol{u}\Big)}.$$

If $X_1, \ldots, X_n$ are **independent** Gaussian variables, then the vector is jointly Gaussian. Indeed,

$$\begin{aligned}\varphi_{\boldsymbol{X}}(\boldsymbol{u}) &= \mathbb{E}\left[\prod_{k=1}^{n} e^{iu_k X_k}\right] \\ &= \prod_{k=1}^{n} \mathbb{E}[e^{iu_k X_k}] \\ &= \exp\left(i\sum_{k=1}^{n} u_k\mu_k - \frac{1}{2}\sum_{k=1}^{n} u_k^2\sigma_k^2\right).\end{aligned}$$

Important qualification. Gaussian marginal distributions alone do not guarantee that a vector is jointly Gaussian. Independence of Gaussian components is sufficient, but without independence or the joint-Gaussian assumption, every component may be normal while some linear combinations are not.

2.2.14 Martingales

Let $(\Omega, \mathcal{F}, (\mathcal{F}_t), \mathbb{P})$ be a filtered probability space. An adapted process $X = (X_t)$ is a **martingale** if

$$\mathbb{E}[|X_t|] < \infty$$

for every t, and, whenever $s \le t$,

$$\boxed{\mathbb{E}[X_t \mid \mathcal{F}_s] = X_s}.$$

Thus, given all information currently available, the conditional mean of a future value is the present value. A martingale is often described as a mathematical model of a fair game.

Suppose the observed value at time s is $X_s = 100$. Many future paths may rise or fall, but their conditional average at a later time t is 100. If new information arrives at a later time u and the realised value is $X_u = 104$, then the updated conditional average of later values becomes 104. Conditional expectations change as the information set grows.

Random-walk example.

Let

$$X_n = \sum_{k=1}^{n} U_k, \qquad \mathbb{P}(U_k = 1) = \mathbb{P}(U_k = -1) = \frac{1}{2},$$

and let

$$\mathcal{F}_n = \sigma(U_1, \ldots, U_n).$$

Because U_{n+1} is independent of $\mathcal{F}_n$ and has mean zero,

$$\begin{aligned}\mathbb{E}[X_{n+1} \mid \mathcal{F}_n] &= \mathbb{E}[X_n + U_{n+1} \mid \mathcal{F}_n] \\ &= X_n + \mathbb{E}[U_{n+1}] \\ &= X_n.\end{aligned}$$

Therefore the simple symmetric random walk is a martingale with respect to its natural filtration.

Submartingales and supermartingales.
An integrable adapted process is a **submartingale** if

$$\mathbb{E}[X_t \mid \mathcal{F}_s] \geq X_s, \qquad s \leq t,$$

and a **supermartingale** if

$$\mathbb{E}[X_t \mid \mathcal{F}_s] \leq X_s.$$

The terminology refers to conditional drift, not to monotone sample paths: a submartingale can fall on individual paths, and a supermartingale can rise.

For a random walk with

$$\mathbb{P}(U_k = 1) = p, \qquad \mathbb{P}(U_k = -1) = 1 - p,$$

the expected increment is $2p - 1$. The process is a submartingale when $p > 1/2$, a martingale when $p = 1/2$, and a supermartingale when $p < 1/2$.

Connection to option pricing.
Under a risk-neutral probability measure $\mathbb{Q}$, it is generally the **discounted** tradable asset price, rather than the undiscounted price, that is modelled as a martingale. With a deterministic short rate r,

$$\widetilde{S}_t = e^{-rt} S_t$$

satisfies

$$\mathbb{E}^{\mathbb{Q}}\left[\widetilde{S}_t \mid \mathcal{F}_s\right] = \widetilde{S}_s, \qquad s \leq t.$$

Equivalently,

$$S_s = e^{-r(t-s)} \mathbb{E}^{\mathbb{Q}}[S_t \mid \mathcal{F}_s].$$

This conditional-expectation principle is central to no-arbitrage derivative pricing.

2.3 Concept Map

Concept	Role
Probability space	Specifies outcomes, measurable events, and their probabilities.
Stochastic process	Provides a time-indexed family of random variables.
Sample path	Shows one realised evolution after an outcome ω is fixed.
Stationary increments	Makes the distribution of a change depend only on elapsed time.
Independent increments	Separates randomness across disjoint intervals.
Filtration	Records the information available through time.
Adaptedness	Ensures the process uses no future information.
MGF and characteristic function	Encode a distribution and, when valid, its moments.
Covariance matrix	Describes the scale and linear dependence of a random vector.
Martingale	Expresses conditional fairness relative to a filtration.

Chapter 3

Markov Chains

Markov chains provide a middle ground between two extreme modelling choices. An i.i.d. model assumes that successive observations are completely independent, while a fully general time-series model may require the entire past to predict the next observation. A Markov chain retains a local form of dependence: the next state depends on the current state, but not on the full history once the current state is known.

3.1 Motivation

3.1.1 Why the Independence Assumption Can Fail

A random variable represents a distribution of possible outcomes rather than one deterministic number. Examples include

$$X \sim \mathcal{N}(\mu, \sigma^2), \qquad Y \sim \text{Bin}(n, p), \qquad Z \sim \text{Bernoulli}(p).$$

Repeatedly drawing from the same distribution is often modelled by an i.i.d. sequence. This assumes both that the distribution remains unchanged and that one observation contains no information about the next. In finance, these assumptions can fail because return distributions drift, volatility changes, and credit or market states persist through time.

An i.i.d. dice-roll portfolio.

Suppose a portfolio value V_t is independently set each day by a fair die:

$$\mathbb{P}(V_t = k) = \frac{1}{6}, \qquad k = 1, \ldots, 6.$$

Then the values are time-invariant and independent. Knowing today's value does not change tomorrow's distribution. This is mathematically convenient, but it cannot express persistence or restrictions on how states evolve.

Loan-delinquency motivation.

Consider four possible states for an individual loan:

$$\mathcal{S} = \{C, D_{30}, D_{60}, D_{90}\},$$

where C denotes current, D_{30} denotes 30–59 days late, D_{60} denotes 60–89 days late, and D_{90} denotes at least 90 days late. If each month's state were drawn independently from a fixed categorical distribution, a current loan could jump directly to D_{90} in one month. That transition is inconsistent with the way delinquency ages.

The empirical proportion of delinquent loans,

$$\widehat{p} = \frac{\text{number of delinquent loans}}{\text{total number of loans}} = \frac{1}{n}\sum_{\ell=1}^{n} \mathbf{1}_{\{\text{loan } \ell \text{ is delinquent}\}},$$

describes a cross-section at one time, but does not describe how individual loans move between states. A transition model is needed.

3.2 Core Structure and Probability Propagation

3.2.1 Definition and Markov Property

Let $X_0, X_1, \ldots$ take values in a finite state space $\mathcal{S} = \{1, \ldots, m\}$. The process is a **Markov chain** if, for every admissible history,

$$\boxed{\mathbb{P}(X_{t+1} = j \mid X_t = i, X_{t-1} = i_{t-1}, \ldots, X_0 = i_0) = \mathbb{P}(X_{t+1} = j \mid X_t = i)}.$$

This is the **Markov property**. It is often called memorylessness, but the name should be interpreted carefully: the chain may be strongly dependent through time. The statement is that the present state contains all the historical information used by the model to predict the next state.

For the loan example,

$$\begin{aligned}&\mathbb{P}(X_{t+1} = D_{60} mid X_t = D_{30}, X_{t-1} = C)\\ &= \mathbb{P}(X_{t+1} = D_{60} \mid X_t = D_{30}).\end{aligned}$$

A chain is **time homogeneous** if the conditional probabilities do not depend on the calendar time:

$$\mathbb{P}(X_{t+1} = j \mid X_t = i) = p_{ij} \quad \text{for every } t.$$

Markov chains may also have countably infinite state spaces, but the notebook and the applications below focus on finite, time-homogeneous chains.

3.2.2 Transition Matrices

The one-step transition probabilities are collected in the matrix

$$P = (p_{ij})_{i,j=1}^{m}, \qquad p_{ij} = \mathbb{P}(X_{t+1} = j \mid X_t = i).$$

Every transition matrix is **row stochastic**:

$$p_{ij} \geq 0, \qquad \sum_{j=1}^{m} p_{ij} = 1 \quad \text{for each } i.$$

The row identifies the current state and the column identifies the next state.

For the delinquency model, the structural zero pattern is

$$P = \begin{array}{c|cccc} & C & D_{30} & D_{60} & D_{90} \\ \hline C & p_{11} & p_{12} & 0 & 0 \\ D_{30} & p_{21} & p_{22} & p_{23} & 0 \\ D_{60} & p_{31} & 0 & p_{33} & p_{34} \\ D_{90} & p_{41} & 0 & 0 & p_{44} \end{array}.$$

Missing arrows or zero entries encode transitions that the model declares impossible in one step.

The numerical matrix used throughout the notebook is

$$P = \begin{pmatrix} 0.99 & 0.01 & 0 & 0 \\ 0.25 & 0.70 & 0.05 & 0 \\ 0.15 & 0 & 0.80 & 0.05 \\ 0.01 & 0 & 0 & 0.99 \end{pmatrix}.$$

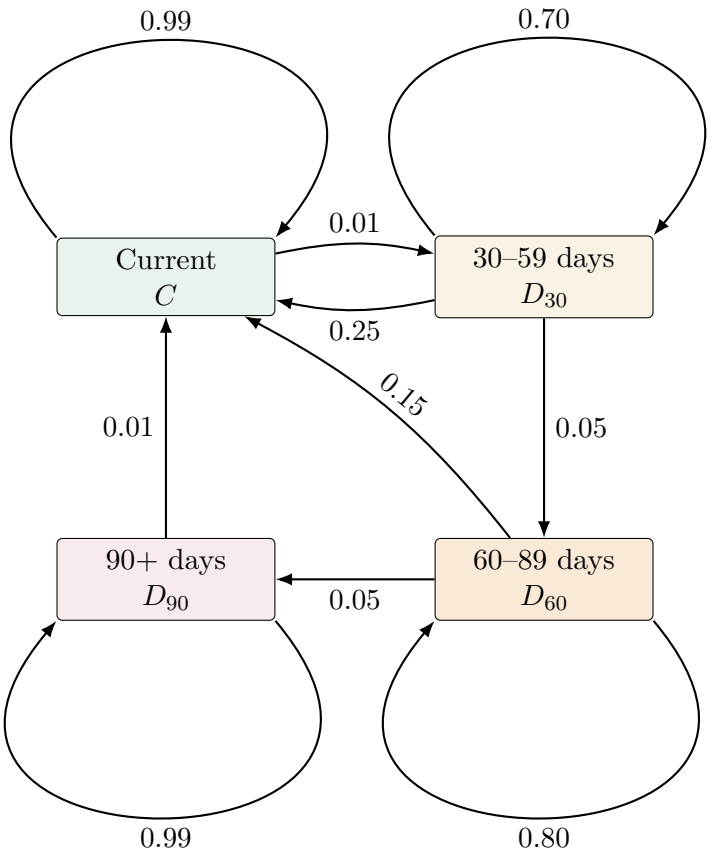


Figure 3.1: The loan-delinquency Markov chain. An omitted arrow represents a zero one-step transition probability.

The matrix describes the evolution of **each loan**, not the portfolio as a single four-state object. The portfolio distribution is obtained by aggregating the state probabilities of its individual loans.

3.2.3 State-Distribution Vectors

Let

$$\boldsymbol{\pi}(t) = \Big(\pi_1(t), \ldots, \pi_m(t)\Big)$$

be a row vector, where

$$\pi_i(t) = \mathbb{P}(X_t = i), \qquad \sum_{i=1}^{m} \pi_i(t) = 1.$$

The one-step evolution follows from the law of total probability:

$$\pi_j(t+1) = \sum_{i=1}^{m} \pi_i(t) p_{ij}.$$

In matrix form,

$$\boxed{\boldsymbol{\pi}(t+1) = \boldsymbol{\pi}(t)P}.$$

For a portfolio of N statistically identical loans, the expected number in state j at time t is

$$\mathbb{E}[N_j(t)] = N\pi_j(t).$$

This expectation formula does not require the loans to be mutually independent, although independence strongly affects portfolio-level uncertainty and confidence intervals.

3.2.4 Chapman–Kolmogorov Equations

Define the n-step transition probability

$$p_{ij}^{(n)} = \mathbb{P}(X_{t+n} = j \mid X_t = i).$$

Conditioning on an intermediate state k gives the **Chapman–Kolmogorov equation**

$$\boxed{p_{ij}^{(a+b)} = \sum_{k=1}^{m} p_{ik}^{(a)} p_{kj}^{(b)}}.$$

Here m is the number of states, while a and b are elapsed numbers of steps. In matrix form,

$$P^{(a+b)} = P^{(a)} P^{(b)}.$$

For a time-homogeneous chain, $P^{(n)} = P^n$, and therefore

$$\boldsymbol{\pi}(t) = \boldsymbol{\pi}(0) P^t.$$

3.2.5 Twelve-Month Loan Transition Probabilities

For the loan matrix above, direct matrix multiplication gives

$$P^{12} \approx \begin{pmatrix} 0.95984 & 0.03168 & 0.00665 & 0.00183 \\ 0.89358 & 0.04109 & 0.03208 & 0.03324 \\ 0.69191 & 0.02031 & 0.07201 & 0.21576 \\ 0.11048 & 0.00272 & 0.00037 & 0.88644 \end{pmatrix}.$$

Thus a currently performing loan has probability

$$\mathbb{P}(X_{12} = D_{90} \mid X_0 = C) = (P^{12})_{C,D_{90}} \approx 0.001826,$$

or approximately 0.1826%, of being at least 90 days delinquent after 12 months under the model.

Starting State	**Current**	**30–59**	**60–89**	**90+**
Current	95.98%	3.17%	0.66%	0.18%
30–59 days	89.36%	4.11%	3.21%	3.32%
60–89 days	69.19%	2.03%	7.20%	21.58%
90+ days	11.05%	0.27%	0.04%	88.64%

For the illustrative initial portfolio distribution

$$\boldsymbol{\pi}(0) = (0.50, 0.20, 0, 0.30),$$

the genuine 12-month distribution is

$$\boldsymbol{\pi}(12) = \boldsymbol{\pi}(0) P^{12} \approx (0.69178, 0.02487, 0.00985, 0.27349).$$

Therefore the expected 90+ delinquency proportion after 12 months is approximately 27.349%. For a portfolio of 1,000 loans, the corresponding expected state counts are approximately

$$(692, 25, 10, 273).$$

Notebook correction. One notebook cell describes its result as a 12-month forecast but computes P^{10000}. Its reported 3.8462% is the long-run stationary 90+ proportion, not the 12-month proportion for the stated initial distribution.

3.3 State Classification and Long-Run Behaviour

3.3.1 Accessibility, Communication, and Irreducibility

A state j is **accessible** from state i, written $i \to j$, if

$$(P^n)_{ij} > 0$$

for some $n \geq 0$. States i and j **communicate** if $i \to j$ and $j \to i$. Communication partitions the state space into communication classes.

A chain is **irreducible** if every pair of states communicates. In an irreducible finite chain, there is no permanently isolated regime: every state can eventually reach every other state with positive probability.

For the loan matrix, the chain is irreducible. For example, $C \to D_{30}$, $D_{30} \to D_{60}$, and $D_{60} \to D_{90}$, while every delinquency state can return to C. Some transitions require several steps even though their one-step probability is zero.

3.3.2 Recurrence, Transience, and Absorption

Starting from state i, define the first return time

$$T_i^+ = \inf\{n \geq 1 : X_n = i\}.$$

The state is **recurrent** if

$$\mathbb{P}_i(T_i^+ < \infty) = 1,$$

and **transient** if this probability is strictly less than 1. A transient state has a positive probability of never being revisited after it is left.

A state i is **absorbing** if

$$p_{ii} = 1.$$

Once the chain enters an absorbing state, it can never leave. The loan matrix has no absorbing state because every diagonal entry is less than 1, although the states C and D_{90} are highly persistent. Because the displayed chain is finite and irreducible, all four states are positive recurrent.

3.3.3 Periodicity

The period of state i is

$$d(i) = \gcd\{n \geq 1 : (P^n)_{ii} > 0\}.$$

A state is **aperiodic** if $d(i) = 1$. In an irreducible chain all states have the same period. A positive self-transition $p_{ii} > 0$ implies aperiodicity for that state; therefore the loan chain is aperiodic because every state has a self-loop.

3.3.4 Stationary Distributions and Ergodicity

A probability row vector $\boldsymbol{\pi}^*$ is a **stationary distribution** if

$$\boxed{\boldsymbol{\pi}^* P = \boldsymbol{\pi}^*, \qquad \sum_{i=1}^{m} \pi_i^* = 1}.$$

If $X_0 \sim \boldsymbol{\pi}^*$, then $X_t \sim \boldsymbol{\pi}^*$ at every later time.

For a finite irreducible and aperiodic chain, the stationary distribution is unique and

$$\boldsymbol{\pi}(0) P^t \longrightarrow \boldsymbol{\pi}^* \qquad \text{as } t \to \infty$$

for every initial distribution. This convergence property is commonly called **ergodicity** in the finite-state setting.

Solving the stationary equations for the loan matrix gives

$$\boldsymbol{\pi}^* = \left(\frac{120}{130}, \frac{4}{130}, \frac{1}{130}, \frac{5}{130}\right) \approx (0.92308, 0.03077, 0.00769, 0.03846).$$

The long-run 90+ delinquency proportion is therefore 3.8462%.

The evolution from the notebook's mixed initial distribution is summarized below.

Month	Current	30–59	60–89	90+
0	50.000%	20.000%	0.000%	30.000%
1	54.800%	14.500%	1.000%	29.700%
2	58.324%	10.698%	1.525%	29.453%
12	69.178%	2.487%	0.985%	27.349%
50	77.115%	2.552%	0.631%	19.701%
∞	92.308%	3.077%	0.769%	3.846%

The convergence is slow because D_{90} has a 99% monthly self-transition probability. Thus 50 months illustrates movement toward the steady state, but does not yet closely approximate it.

Steady-state warning. The limiting distribution is meaningful only if the transition mechanism remains stable. Economic cycles, seasonality, policy changes, underwriting changes, and external shocks can make the true transition matrix vary through time.

3.4 Model Design and Estimation

3.4.1 Choosing States in Quantitative Finance

State definitions are problem dependent. Common choices include

- **volatility regimes:** low, medium, and high, using a measure such as the VIX or realised volatility;
- **market trends:** bullish, bearish, and sideways, using index returns or trend indicators;
- **liquidity regimes:** tight, normal, and wide spreads, using quoted or effective bid–ask spreads;
- **credit states:** current and progressively older delinquency buckets.

Thresholds should be chosen before estimating transitions, or selected using a properly separated training sample. Poor thresholds can discard important information or create states with too few observed transitions.

3.4.2 Maximum Likelihood Estimation: General Idea

Maximum likelihood estimation chooses the parameter value under which the observed data have the greatest probability or density. For example, if $Y_1, \ldots, Y_n$ are modelled as i.i.d. $\mathcal{N}(\mu, \sigma^2)$, then maximizing the Gaussian likelihood gives

$$\widehat{\mu} = \frac{1}{n}\sum_{k=1}^{n} Y_k, \qquad \widehat{\sigma}^2 = \frac{1}{n}\sum_{k=1}^{n}(Y_k - \widehat{\mu})^2.$$

As more representative observations become available, these estimates typically stabilize near the data-generating parameters. Early estimates can be volatile because they are based on small samples.

The same principle estimates a Markov transition matrix: instead of fitting the parameters of one density, we fit every row of conditional transition probabilities.

3.4.3 Deriving the Markov-Chain MLE

Suppose the observed state sequence is

$$x_0, x_1, \ldots, x_T,$$

and let $\nu_i = \mathbb{P}(X_0 = i)$ be the initial distribution. Under the time-homogeneous Markov model, the likelihood is

$$L(P) = \nu_{x_0} \prod_{t=0}^{T-1} p_{x_t x_{t+1}}.$$

Define the transition counts

$$n_{ij} = \sum_{t=0}^{T-1} \mathbf{1}_{\{x_t = i, x_{t+1} = j\}}, \qquad n_i = \sum_{j=1}^{m} n_{ij}.$$

Then

$$L(P) = \nu_{x_0} \prod_{i=1}^{m}\prod_{j=1}^{m} p_{ij}^{n_{ij}},$$

and, ignoring the initial term when it does not involve P,

$$\ell(P) = \log L(P) = \text{constant} + \sum_{i=1}^{m}\sum_{j=1}^{m} n_{ij} \log p_{ij}.$$

Each row is maximized subject to

$$\sum_{j=1}^{m} p_{ij} = 1, \qquad p_{ij} \geq 0.$$

For row i, use the Lagrangian

$$\mathcal{L}_i = \sum_{j=1}^{m} n_{ij} \log p_{ij} + \lambda_i \left(1 - \sum_{j=1}^{m} p_{ij}\right).$$

The first-order condition is

$$\frac{\partial \mathcal{L}_i}{\partial p_{ij}} = \frac{n_{ij}}{p_{ij}} - \lambda_i = 0,$$

so $p_{ij} = n_{ij}/\lambda_i$. Summing over j yields $\lambda_i = n_i$, and therefore

$$\boxed{\widehat{p}_{ij} = \frac{n_{ij}}{n_i} = \frac{\text{observed transitions } i \to j}{\text{all observed departures from } i}}.$$

The MLE is simply the observed transition frequency within each row. If $n_i = 0$, that row cannot be estimated from the data without additional structure or smoothing.

For multiple loan histories, the counts n_{ij} are aggregated across all loans and all adjacent observation times. The chain is fitted to each loan's state transitions, not directly to the aggregate portfolio proportions.

3.4.4 Loan-Portfolio MLE Example

The notebook generates 1,000 loan histories over 12 months using the matrix

$$P = \begin{pmatrix} 0.99 & 0.01 & 0 & 0 \\ 0.25 & 0.70 & 0.05 & 0 \\ 0.15 & 0 & 0.80 & 0.05 \\ 0.01 & 0 & 0 & 0.99 \end{pmatrix}$$

and an initial distribution

$$(0.85, 0.08, 0.05, 0.02).$$

One simulated cross-section contains 865 current loans, 67 loans 30–59 days late, 46 loans 60–89 days late, and 22 loans at least 90 days late. Sampling variation explains why these counts are not exactly equal to their expected values $(850, 80, 50, 20)$.

Aggregating observed transitions gives the MLE

$$\widehat{P} \approx \begin{pmatrix} 0.99198 & 0.00802 & 0 & 0 \\ 0.23325 & 0.71712 & 0.04963 & 0 \\ 0.15504 & 0 & 0.77132 & 0.07364 \\ 0.00545 & 0 & 0 & 0.99455 \end{pmatrix}.$$

The estimates are close to the generating matrix. Rows or transitions with fewer observations show greater sampling error; for example, the estimated $D_{60} \to D_{90}$ probability differs more noticeably from its true value than the frequently observed current-state probabilities.

3.4.5 Assumptions Behind the Estimator

The interpretation and large-sample reliability of $\widehat{P}$ depend on several assumptions.

1. **Markov property.** The current state contains all past information relevant to the next transition.
2. **Time homogeneity.** The matrix P is constant across the sample and forecast horizon.
3. **Sufficient state visitation.** Every row of interest has enough departures, and rare transitions are observed often enough to be estimated.
4. **Ergodicity for one long history.** The chain must explore its relevant states so empirical transition frequencies converge to stable probabilities.
5. **Cross-sectional independence or an appropriate dependence model.** Treating different loans as independent ignores shared exposure to unemployment, interest rates, geography, underwriting standards, and other macroeconomic shocks.

Time homogeneity and cross-loan independence are often the most questionable assumptions in credit applications. Violating them can make standard errors too small and long-horizon forecasts misleading.

3.4.6 Large-Sample Properties of the MLE

Under suitable regularity, identifiability, and ergodicity conditions, the transition-frequency estimator has the following properties.

- **Consistency:** $\widehat{p}_{ij} \to p_{ij}$ as the number of observed departures from state i grows.
- **Asymptotic normality:** conditional on n_i, row i behaves like a multinomial estimate. Writing $\boldsymbol{p}_i = (p_{i1}, \ldots, p_{im})^{\mathsf{T}}$,

$$\sqrt{n_i}(\widehat{\boldsymbol{p}}_i - \boldsymbol{p}_i) \xrightarrow{d} \mathcal{N}\Big(\mathbf{0}, \operatorname{diag}(\boldsymbol{p}_i) - \boldsymbol{p}_i\boldsymbol{p}_i^{\mathsf{T}}\Big).$$

- **Asymptotic efficiency:** under standard conditions, the MLE achieves the usual information-bound efficiency within the specified model.
- **Invariance:** if $g(P)$ is a function of the transition parameters, its MLE is $g(\widehat{P})$. For example, an estimated n-step matrix is $\widehat{P}^{,n}$.

The law of large numbers explains why empirical probabilities tend to move toward their true values as the data set grows. It does not guarantee that a misspecified Markov model converges to the real economic mechanism.

> **MLE versus maximum entropy.** Maximum likelihood and maximum entropy are distinct principles. The MLE maximizes the likelihood of the observed data. It is not generally a maximum-entropy estimator unless a separate entropy argument and appropriate constraints establish that connection.

3.5 Limitations and Extensions

The Markov assumption is a substantial improvement over unconditional independence, but it remains a simplification. Important limitations include

- dependence on events further in the past than the current state;
- transition probabilities that vary with economic conditions;
- common shocks that couple many loans or assets;
- imperfect, noisy, or arbitrarily discretized state definitions;
- unobserved regimes that influence the observed process.

Natural extensions include higher-order Markov chains, non-homogeneous transition matrices, covariate-dependent transition models, regime-switching models, and hidden Markov models. In quantitative finance, Markov chains are often the first step toward models of volatility regimes, liquidity regimes, credit migration, and latent market states.

> **Core modelling lesson.** Full independence is frequently too strong. Markov chains replace it with local conditional dependence, enabling probabilities and expectations to be propagated with a transition matrix. The resulting model is useful only to the extent that its states, transition rules, and stability assumptions reflect the real system.

Chapter 4

Random Walk

A random walk is a concrete discrete-time stochastic process and, when its current position is taken as the state, a Markov chain. The simple symmetric case below turns the preceding abstract ideas into explicit paths, probabilities, and barrier calculations before leading naturally to Brownian motion.

4.1 Simple Symmetric Random Walk

4.1.1 Definition and Rule

A **simple symmetric random walk** is one of the simplest stochastic processes. At every discrete time step, there is:

- a 50% probability of moving $+1$, one unit to the right;
- a 50% probability of moving -1, one unit to the left.

Update rule.

$$x(n+1) = x(n) \pm 1.$$

Each outcome, $+1$ or -1, is equally likely; this is why the walk is called symmetric.

4.1.2 Coin-Flip Analogy

Coin-Flip Outcome	Movement
Heads	Move $+1$
Tails	Move -1

4.1.3 The Growing Tree of Possible Positions

Time n	Possible Positions
0	$\{0\}$
1	$\{-1, +1\}$
2	$\{-2, 0, +2\}$
3	$\{-3, -1, +1, +3\}$

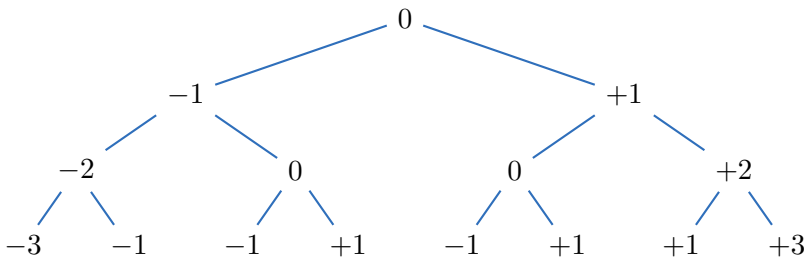


Figure 4.1: The first three steps of a simple symmetric random walk.

Key intuition. As time progresses, the number of possible paths and reachable positions increases, making the path increasingly unpredictable even though the movement rule remains simple. In the limit of many suitably scaled small steps, this behaviour leads to Brownian motion.

4.2 Counting Random-Walk Paths

The combinatorial approach to random walks treats every sequence of upward and downward steps as a path. This viewpoint is especially useful because, when the steps are independent and equally likely, calculating a probability reduces to counting the paths in the corresponding event.

4.2.1 Paths Between Two Space–Time Points

Let

$$\alpha = (m, p), \qquad \beta = (n, q), \qquad m < n,$$

be two points in the space–time plane. A random-walk path from α to β is a function

$$\pi : \{m, m+1, \ldots, n\} \longrightarrow \mathbb{Z}$$

such that

$$\pi(m) = p, \qquad \pi(n) = q, \qquad |\pi(k+1) - \pi(k)| = 1.$$

Thus each step changes the position by exactly one unit. Denote the set of all such paths by

$$\mathrm{PATH}(\alpha, \beta).$$

For paths starting at the origin, define

$$N_{n,x} := \Big|\mathrm{PATH}\Big((0,0),(n,x)\Big)\Big|.$$

4.2.2 Total Number of Paths

At each of n time steps there are two possible moves, $+1$ or -1. Consequently, the total number of length-n paths starting at the origin is

$$2^n.$$

Under the simple symmetric random-walk model, every one of these paths has probability 2^{-n}.

4.2.3 Number of Paths to a Fixed Endpoint

Suppose a path reaches position x after n steps. If u is the number of upward steps and d the number of downward steps, then

$$u + d = n, \qquad u - d = x.$$

Solving these equations gives

$$u = \frac{n+x}{2}, \qquad d = \frac{n-x}{2}.$$

The path is determined once the u time indices containing upward steps are selected. Hence

$$N_{n,x} = \begin{cases} \binom{n}{(n+x)/2}, & |x| \le n \text{ and } n+x \text{ is even}, \\ 0, & \text{otherwise.} \end{cases}$$

The parity condition is important: after an even number of steps the walk can reach only even positions, while after an odd number of steps it can reach only odd positions. Dividing the path count by 2^n gives the endpoint distribution

$$\boxed{\mathbb{P}(X_n = x) = 2^{-n} N_{n,x} = 2^{-n} \binom{n}{(n+x)/2}},$$

whenever the endpoint is reachable.

Example: returning to the origin after ten steps.
To finish at the starting position, the path must contain five upward and five downward steps. Therefore

$$\mathbb{P}(X_{10} = 0) = \frac{N_{10,0}}{2^{10}} = \frac{\binom{10}{5}}{2^{10}} = \frac{252}{1024} = \frac{63}{256} \approx 0.2461.$$

4.2.4 A Useful Recursion

Every path ending at (n, x) must arrive on its final step either from $(n-1, x-1)$ or from $(n-1, x+1)$. The two possibilities are disjoint, so

$$N_{n,x} = N_{n-1,x-1} + N_{n-1,x+1}, \qquad N_{0,0} = 1.$$

This recurrence is the same addition rule that generates Pascal's triangle, which explains the appearance of binomial coefficients in the endpoint formula.

4.2.5 Translation Invariance

Shifting a path horizontally or vertically does not change its pattern of upward and downward steps. Therefore

$$\Big|\mathrm{PATH}\big((a,p),(b,q)\big)\Big| = N_{b-a,q-p}.$$

Only the elapsed time $b - a$ and the net displacement $q - p$ matter.

Example.
A path from $(2,3)$ to $(15,2)$ has length 13 and displacement -1. Translating its starting point to the origin gives

$$\left|\mathrm{PATH}\Big((2,3),(15,2)\Big)\right| = N_{13,-1} = \binom{13}{6} = 1716.$$

4.3 Return Probabilities

A simple symmetric random walk can return to the origin only after an even number of steps. At time $2n$, exactly n steps must be upward and n must be downward, so

$$\boxed{\mathbb{P}(X_{2n}=0) = \frac{\binom{2n}{n}}{2^{2n}}}.$$

For $n=5$, this reproduces the ten-step probability $63/256$. Stirling's approximation further gives

$$\mathbb{P}(X_{2n}=0) \sim \frac{1}{\sqrt{\pi n}} \qquad \text{as } n \to \infty.$$

Thus the probability of being at the origin at one particular late time decreases to zero, but only at the relatively slow rate $n^{-1/2}$.

4.4 The Reflection Principle

Path counting becomes more subtle when a path is required to touch, cross, or avoid a barrier. The reflection principle converts such a constrained count into an unconstrained endpoint count.

Let L be the horizontal line at integer height h, and let

$$\alpha = (m,p), \qquad \beta = (n,q),$$

lie strictly on the same side of L. Reflect α across L to obtain

$$\alpha' = (m, 2h-p).$$

Reflection principle. The number of paths from α to β that touch or cross L is equal to the total number of unrestricted paths from the reflected point α' to β:

$$\#\{\alpha \to \beta \text{ paths meeting } L\} = \left|\mathrm{PATH}(\alpha',\beta)\right|.$$

Using translation invariance, this count can be written explicitly as

$$|\mathrm{PATH}(\alpha',\beta)| = N_{n-m,q-(2h-p)} = N_{n-m,q-2h+p}.$$

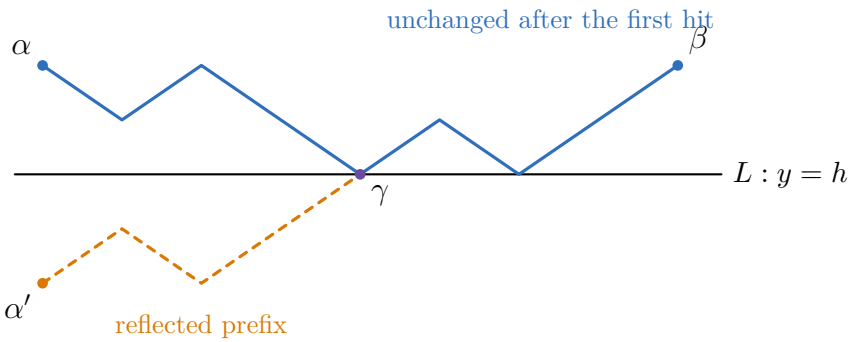


Figure 4.2: Reflecting the path up to its first meeting point γ with the barrier produces a path from α' to β.

4.4.1 Why the Reflection Principle Works

Let $\mathcal{S}$ be the set of paths from α to β that meet L. For any $\pi \in \mathcal{S}$, let

$$\tau = \min\{k : \pi(k) = h\}$$

be the first time the path meets the barrier. Define a new path by reflecting the part before this first meeting:

$$(f\pi)(k) = \begin{cases} 2h - \pi(k), & m \le k \le \tau, \\ \pi(k), & \tau \le k \le n. \end{cases}$$

The transformed path starts at α', passes through the same first meeting point $\gamma = (\tau, h)$, and then follows the original path to β.

This construction is a bijection:

1. Every constrained path produces a path from α' to β.
2. Since α' and β lie on opposite sides of L, every path between them must meet L.
3. Reflecting the prefix of that path up to its first meeting with L recovers the unique original path.

Therefore the constrained and reflected path sets have the same size.

4.4.2 Counting Paths That Avoid a Barrier

The total number of paths from $\alpha = (m, p)$ to $\beta = (n, q)$ is $N_{n-m,q-p}$. Subtracting the number that meet L gives

$$\boxed{\#\{\alpha \to \beta \text{ paths avoiding } L\} = N_{n-m,q-p} - N_{n-m,q-2h+p}}.$$

This formula is the basic tool for questions about positivity, maximum levels, and first-passage events.

Example: paths that remain strictly positive.

Count the ten-step paths from $(0, 0)$ to $(10, 4)$ that satisfy $X_k > 0$ for every $k = 1, \ldots, 10$. The first step must be upward, so the problem reduces to paths from $(1, 1)$ to $(10, 4)$ that

avoid the line $L : y = 0$. Thus

$$\begin{aligned}\#\{\text{strictly positive paths ending at } 4\} &= N_{9,3} - N_{9,5} \\ &= \binom{9}{6} - \binom{9}{7} \\ &= 84 - 36 \\ &= 48.\end{aligned}$$

Since there are 2^{10} total ten-step paths, the corresponding unconditional probability is

$$\frac{48}{2^{10}} = \frac{3}{64}.$$

4.5 Interpretation and Limitations in Finance

A scaled random walk can be used as a first model of price changes:

$$S_n = S_0 + aX_n,$$

where a is the monetary size of one price movement. The model captures the idea that short-term changes are uncertain and that many possible price paths may emerge from the same initial value. It also gives exact formulas for endpoint probabilities and barrier events.

However, the simple symmetric model is not a realistic complete model of an asset price:

- it assumes equal probabilities of upward and downward moves;
- it assumes independent increments of constant size;
- it permits negative values when applied directly to prices;
- it does not account for changing volatility, jumps, or market microstructure.

These limitations motivate the continuous-time Brownian-motion and geometric Brownian-motion models developed in the following sections.

4.6 Random Walk as the Discrete Precursor to Brownian Motion

Let $\boldsymbol{X} = (X_0, X_1, X_2, \ldots)$ be the symmetric simple random walk defined by

$$X_n = \sum_{i=1}^{n} U_i,$$

where $\boldsymbol{U} = (U_1, U_2, \ldots)$ is a sequence of independent random variables satisfying

$$\mathbb{P}(U_i = 1) = \mathbb{P}(U_i = -1) = \frac{1}{2} \qquad \text{for every } i.$$

Key statistical properties.

$$\mathbb{E}[X_n] = 0, \qquad \mathrm{Var}(X_n) = n.$$

Because X is the partial-sum process of an i.i.d. sequence, it has **stationary, independent increments**, although only in discrete time.

Central Limit Theorem connection.

$$\frac{X_n}{\sqrt{n}} \xrightarrow{d} \mathcal{N}(0,1) \qquad \text{as } n \to \infty.$$

Constructing a continuous-time process.

For $h, d \in (0, \infty)$, define the continuous-time jump process

$$X_{h,d}(t) = dX_{\lfloor t/h \rfloor}.$$

Its jumps occur at $\{0, h, 2h, \ldots\}$, and every jump has size $\pm d$.

We want to let $h \downarrow 0$, producing smaller time steps, and $d \downarrow 0$, producing smaller spatial jumps, simultaneously. These limits cannot be chosen arbitrarily because

$$\mathbb{E}[X_{h,d}(t)] = 0, \qquad \mathrm{Var}[X_{h,d}(t)] = d^2 \left\lfloor \frac{t}{h} \right\rfloor \approx \frac{d^2 t}{h}.$$

The critical scaling.

If $d = \sqrt{h}$, the Central Limit Theorem implies that the distribution of $X_{h,d}(t)$ converges to a normal distribution with mean 0 and variance t as $h \downarrow 0$:

$$\boxed{d = \sqrt{h} \quad \Longrightarrow \quad X_{h,d}(t) \xrightarrow{d} \mathcal{N}(0,t)}.$$

The mathematical bridge. When the random walk's spatial step is the square root of its time step, the scaled discrete process converges to standard Brownian motion. Its limiting paths are continuous and appear smooth at a distance, but are almost surely nowhere differentiable.

Chapter 5

Derivatives

5.1 Brownian Motion

5.2 Modelling Stock Price

5.3 Itô's Lemma and Girsanov's Theorem

5.4 Factors Affecting Option Prices

5.5 Black–Scholes

5.6 Deriving the Black–Scholes Equation

5.7 Options Pricing

5.8 Binomial Model

5.9 Option Greeks

Chapter 6

Machine Learning in Financial Markets

6.1 Sentiment Analysis: Opinion Mining

6.2 Research Papers

6.3 Datasets for Self-Practice

6.4 Time Series Analysis: Handling Volatility

6.5 Sequence-to-Sequence Modelling in Finance